A Hybrid DAG-Based Provenance and Blockchain Anchoring Framework for Integrity Verification in Academic Research Workflows

INDIVIDUAL THESIS

Darshan R

IT22097156

Individual component: Hybrid DAG-Based Provenance and Blockchain Anchoring for Academic Research Workflows

Group ID: R26-SE-023

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Darshan R

Student ID: IT22097156

Signature: R.Darshan Date: 11/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Jeewaka Perera		
Co-supervisor: Poojani Gunathilake		

ABSTRACT

Research management platforms store group registrations, topic decisions, reviews and submissions in ordinary databases, where a privileged user can alter an academic record without leaving independent evidence. Public blockchains can provide that evidence, but their latency, cost, privacy exposure and rigid contract logic make them a poor home for a dynamic, student-facing workflow. This thesis presents the Integrity and Governance component of ResearchLens, the platform of the AI-Powered Research Gap Identifier & Trend Analyzer project. The component gives academic events tamper-evident provenance and public proof without putting academic data on-chain or making users wait for the blockchain. A review of 36 studies on blockchain provenance, platform performance, smart contract security and document similarity identified six gaps that shaped the design. Each academic event is hashed as a canonical snapshot of its database record and written to a local directed acyclic graph (DAG) ledger made of group branches tied to a main checkpoint chain. The event block's hashes are then anchored on Ethereum Sepolia through a minimal, owner-only Solidity contract. Local attestation is immediate, while a background reconciler confirms anchors every 60 seconds. The service, built with FastAPI, PostgreSQL and Web3.py, also rejects duplicate files by SHA-256 hash and scores semantic similarity with all-MiniLM-L6-v2, which was chosen after a comparison of three embedding models. Six test cases on the deployed system confirmed the design. A new group was confirmed on Sepolia about 2 minutes 15 seconds after registration without delaying the user, and direct edits to an academic record, a ledger block and an anchor link were each detected and reported with a distinct status. A duplicate file was refused, and a similar but non-identical submission was flagged for manual review. The work shows that blockchain serves academic governance best as an external trust anchor rather than as the primary database.

Keywords: Blockchain anchoring, data provenance, acyclic graph ledger, academic integrity, Ethereum Sepolia

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Jeewaka Perera, and my co-supervisor, Poojani Gunathilake, for their guidance, constructive feedback and encouragement throughout this research. I also thank the members of my research group for their collaboration in developing the research management platform with which this component is integrated, and the academic and technical staff of the Department of Information Technology at the Sri Lanka Institute of Information Technology for their support.

I am grateful to the authors of the open-source tools and public infrastructure on which this work depends, including the Ethereum Sepolia test network. Finally, I thank my family and friends for their patience and support during this project.

TABLE OF CONTENTS

1 INTRODUCTION..... 1

1.1 Overall project context..... 1

1.2 Individual component context..... 2

1.3 Problem statement..... 3

1.4 Research gap..... 3

1.5 Aim and objectives..... 4

1.6 Research questions or hypotheses..... 5

1.7 Scope limitations and assumptions..... 5

1.8 Individual contribution..... 6

1.9 Thesis structure..... 6

2 LITERATURE REVIEW AND RELATED WORK..... 8

2.1 Review method and scope..... 8

2.2 Domain and theoretical foundations..... 9

2.3 Related research..... 11

2.4 Existing systems products and techniques..... 14

2.5 Gap synthesis and conceptual framework..... 15

3 RESEARCH METHODOLOGY..... 18

3.1 Research design..... 18

3.2 Data participants or study materials..... 19

3.3 Experimental or study procedure..... 20

3.4 Measures and evaluation metrics..... 21

3.5 Data analysis..... 21

3.6 Ethics privacy security and responsible AI..... 22

3.7 Reproducibility and research artefacts..... 23

4 PRODUCT AND SOLUTION DESIGN 24
4.1 Requirements 24
4.2 Component architecture within the complete solution 25
4.3 Design alternatives and decisions 27
4.4 Detailed design 29
4.5 Standards and professional practice 35
5 IMPLEMENTATION INTEGRATION AND TESTING 37
5.1 Development environment and technology stack 37
5.2 Component implementation 38
5.3 Integration with the other components 40
5.4 Verification and software testing 41
5.5 Deployment and operation 42
6 RESULTS AND VALIDATION 43
6.1 Research results 43
6.2 Product and technical results 43
6.3 User or stakeholder validation 50
6.4 Integration results 50
6.5 Summary of findings 51
7 DISCUSSION 52
7.1 Interpretation of findings 52
7.2 Comparison with literature baselines and existing solutions 53
7.3 Research contribution 54
7.4 Product contribution 55
7.5 Limitations and threats to validity 55
7.6 Implications 56
8 COMMERCIALISATION AND SUSTAINABILITY 58

8.1 Value proposition and target users.....58
8.2 Market and competitive position.....58
8.3 Business or sustainability model.....58
8.4 Scalability intellectual property and compliance.....59
8.5 Contribution to the Sustainable Development Goals.....60
9 CONCLUSION AND FUTURE DIRECTIONS.....61
9.1 Conclusion.....61
9.2 Achievement of objectives.....62
9.3 Individual research and product contributions.....62
9.4 Future directions.....62
REFERENCES.....64
GLOSSARY.....69
APPENDICES.....70
Appendix A Research instruments or extended results.....70
Appendix B Academic integrity and AI use disclosure.....72

LIST OF FIGURES

Figure 2.1: Conceptual framework linking the identified gaps to the design responses of the component..... 17

Figure 4.1: Layered architecture of the complete solution, with the integrity governance component highlighted.....26

Figure 4.2: Illustrative DAG ledger with a main checkpoint chain and two group branches (node labels show proof state).....31

Figure 4.3: Attestation flow separating synchronous local attestation from asynchronous blockchain confirmation..... 34

Figure 4.4: State model of the attestation queue and of blockchain anchors..... 35

Figure 6.1: Integrity DAG in the administrator workspace, showing group branches and the proof state of each event (TC1)..... 44

Figure 6.2: Integrity Audit Trail with local validity, Sepolia status and block numbers per event..... 45

Figure 6.3: Rejection of a previously submitted proposal file (TC2).....45

Figure 6.4: TAF submission returned with a similarity score and a manual-review decision (TC3)..... 47

Figure 6.5: Topic block of R26-SE-002 marked Invalid after a direct change of supervisor (TC4).....48

Figure 6.6: Audit Trail after one character of block 45's hash was changed (TC5)... 48

Figure 6.7: TAF of R26-SE-004 shown as Not anchored after its anchor link was altered (TC6).....49

LIST OF TABLES

Table 2.1: Review themes, search focus and number of extracted records..... 8

Table 2.2: Gap synthesis and corresponding design response.....16

Table 3.1: Evaluation measures and their relation to the research questions..... 21

Table 4.1: Functional requirements of the integrity governance component..... 24

Table 4.2: Non-functional requirements of the integrity governance component.....25

Table 4.3: Placement of data between off-chain storage and the blockchain.....26

Table 4.4: Comparison of a local blockchain-inspired ledger with a fully on-chain Ethereum application.....27

Table 4.5: Academic event types attested by the component..... 29

Table 4.6: Payload keys of academic event blocks and main checkpoint blocks.....29

Table 4.7: Node proof-state labels in the DAG visualisation..... 31

Table 4.8: Interpretation of combined local and Sepolia status.....32

Table 4.9: Fields of an on-chain anchor record..... 32

Table 5.1: Software versions declared by the Docker deployment of the integrity service.....37

Table 5.2: Principal endpoints of the integrity service..... 38

Table 5.3: Configuration items of the integrity service..... 38

Table 5.4: Test cases and outcomes..... 41

Table 6.1: Summary of results for each research question..... 43

Table 6.2: Comparison of sentence-embedding models on the 22-document corpus 46

Table 6.3: Observed behaviour of synchronous and asynchronous anchoring.....49

Table 6.4: Integration results..... 50

Table 7.1: Comparison with representative blockchain provenance systems.....53

Table 8.1: Contribution to the Sustainable Development Goals.....60

Table 9.1: Achievement of objectives..... 62

Table A.1: Records extracted in the literature review..... 70

Table A.2: Index of validation evidence..... 72

Table B.1: Generative AI and automated tool use..... 73

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ABI	Application binary interface
API	Application programming interface
BERT	Bidirectional Encoder Representations from Transformers
CNN	Convolutional neural network
DAG	Directed acyclic graph
FR	Functional requirement
IoMT	Internet of Medical Things
IoT	Internet of Things
IPFS	InterPlanetary File System
JSON	JavaScript Object Notation
NBSVM	Naive Bayes–support vector machine
NFR	Non-functional requirement
PoA	Proof of authority
PoS	Proof of stake
PoW	Proof of work
REST	Representational state transfer
RPC	Remote procedure call
RQ	Research question
SBERT	Sentence-BERT
SDG	Sustainable Development Goal
SHA-256	Secure Hash Algorithm with a 256-bit digest
TAF	Topic assessment form

1 INTRODUCTION

1.1 Overall project context

Universities increasingly manage final-year and postgraduate research through digital platforms rather than paper forms and e-mail threads. In such platforms, students register research groups, propose topics, upload topic assessment forms (TAFs), submit project proposals, request supervisor approval, revise topics, upload checklist documents and submit milestone deliverables such as final reports, websites, research papers, theses and publication evidence. Supervisors review these submissions, record decisions and comments, and approve or reject topic changes, while administrators configure the workflow and its requirements. Moving these activities online improves convenience, traceability of deadlines and coordination between students and supervisors.

This thesis reports one individual component of the group research project *AI-Powered Research Gap Identifier & Trend Analyzer* (group R26-SE-023), whose working platform is called ResearchLens. ResearchLens supports the whole research lifecycle for students, supervisors and administrators. It has a React and Vite frontend, a Node.js and Express workflow backend, and a PostgreSQL database that holds groups, topics, submissions, reviews, checklist requirements and related records; uploaded documents are kept in backend file storage.

The project is divided into four individual components. The first, Semantic Analysis and Gap Detection, analyses research literature to surface gaps. The second, Recommendation and Collaboration Network, recommends supervisors, collaborators and related work. The third, Collaborative Workspace and Progress Tracking, supports day-to-day group work and milestone tracking. The fourth, Integrity and Governance, is the subject of this thesis. It sits underneath the other three and records the academic events they produce in a tamper-evident way.

Although digitalisation improves efficiency, it introduces a governance concern. The academic record of a research project, including who submitted what, when it was submitted, which supervisor approved it and how the topic evolved, is normally held in mutable database tables and file systems. If a submission, approval decision, topic

title, document or review note is altered after the fact, a conventional system may provide no independent evidence of the original state. The individual component described in this thesis addresses this concern.

1.2 Individual component context

The individual component is an **integrity governance module** that provides tamper-evident provenance and public integrity proof for the academic events recorded by the platform. It runs as a separate FastAPI service, deployed in Docker, which the workflow backend calls whenever a significant academic event occurs. For each event, the module reconstructs a canonical snapshot of the relevant database record, computes a SHA-256 hash of that snapshot, and appends a hash-linked block to a local provenance ledger. The ledger is organised as a directed acyclic graph (DAG) consisting of a main checkpoint chain and one branch per research group. The block hash and record hash of each academic event block are then anchored on the Ethereum Sepolia public test network through a minimal Solidity smart contract, while all documents and academic metadata remain off-chain.

The module also verifies the ledger on demand. Verification recomputes record hashes from the live database and compares them with the hashes stored at attestation time, and it checks that the corresponding anchors exist on Sepolia. The results are presented to administrators through an Integrity DAG view and an Integrity Audit Trail, which show, for every event, whether it is locally valid and whether its public proof is on-chain. A background reconciler confirms pending transactions, so students and supervisors never wait for the blockchain.

The same service also guards document submissions. It rejects a file that has already been submitted by comparing SHA-256 file hashes, and it scores semantic similarity against earlier submissions using sentence embeddings (all-MiniLM-L6-v2) and FAISS vector search. Similarity supports academic judgement, while hashing and anchoring provide tamper evidence; the two are deliberately kept apart.

1.3 Problem statement

Academic research governance depends on a trustworthy history of lifecycle events. A group must exist before a topic can be submitted; a TAF may be rejected and

resubmitted; a proposal may request a topic change; a supervisor approval may change the effective topic for all later submissions; and checklist and milestone documents occur between presentation stages. Disputes about authorship, supervision, originality and approval are difficult to resolve if this history is incomplete or can be silently modified.

Conventional systems record provenance through database timestamps, audit columns and application logs. These mechanisms are useful, but they depend on the trustworthiness of the database administrator and the application server: a privileged actor who modifies both a record and its log can leave no detectable trace. Public blockchains offer an alternative because they provide append-only, tamper-resistant and externally verifiable ledgers [1], [2]. However, placing an entire academic workflow on a public blockchain is impractical. Documents are large, academic data is sensitive, smart contracts are difficult to modify after deployment, and public confirmation can be slow and unpredictable [3]–[5]. In a student-facing system, waiting for blockchain confirmation inside a request can make the user interface appear frozen.

The problem addressed by this thesis is therefore stated as follows: *how can an academic research management workflow provide tamper-evident provenance and public integrity proof without storing private academic data on-chain or blocking users on blockchain confirmation latency?*

1.4 Research gap

Blockchain-based provenance and integrity verification have been studied extensively in healthcare, the Internet of Medical Things (IoMT), cloud storage, energy trading and multi-stakeholder data governance [6]–[15]. Many of these works adopt hybrid designs that keep original data off-chain and store hashes or metadata on-chain. None of the reviewed studies, however, models the university research lifecycle, with its ordered and role-dependent events, group-level branching, resubmissions and topic evolution. Existing systems typically record file hashes rather than event-level academic accountability, use a single linear chain that hides concurrent group progress, assert that immutability guarantees integrity without

simulating tampering against live database state, and treat blockchain latency as an acceptable cost rather than as a usability problem for interactive workflows. These gaps are analysed in detail in Section 2.5.

1.5 Aim and objectives

The aim of this research is to design, implement and validate a hybrid integrity governance component that provides tamper-evident, publicly verifiable provenance for academic research workflow events while keeping academic data private and user workflows responsive.

The specific objectives are:

1. To review the literature on blockchain-based provenance, blockchain platform performance, smart contract security and document similarity in order to identify reusable design patterns, known limitations and research gaps relevant to academic workflow integrity (O1).
2. To design an event-level provenance model for the academic research lifecycle that represents both the overall ledger timeline and group-specific progress, using a DAG composed of a main checkpoint chain and group-wise branches (O2).
3. To implement canonical record snapshot hashing and two-level ledger verification that detect modification both of ledger blocks and of the underlying database records after attestation (O3).
4. To design and deploy a minimal, owner-restricted and non-upgradeable Solidity smart contract that anchors cryptographic proofs on the Ethereum Sepolia testnet without storing documents or private academic content (O4).
5. To separate synchronous local attestation from asynchronous blockchain confirmation through broadcast-and-reconcile anchoring, so that academic operations are not blocked by public confirmation latency (O5).
6. To integrate the component with the platform and its administrative views, and to validate it through test cases that include deliberate tampering with academic records, ledger blocks and anchor links, together with duplicate-file and semantic-similarity checks on document submissions (O6).

1.6 Research questions or hypotheses

The main research question follows directly from the problem statement:

Main research question. How can an academic research management workflow provide tamper-evident provenance and public integrity proof without storing private academic data on-chain or blocking users on blockchain confirmation latency?

The main question is decomposed into four sub-questions:

1. RQ1: How can academic research lifecycle events be modelled in a provenance ledger so that both the overall timeline and the independent progress of each research group are represented and verifiable?
2. RQ2: How can modification of an attested academic record be detected, including modification made directly in the database by a privileged actor?
3. RQ3: How can external, public proof of the attested state be obtained while keeping documents and academic metadata off-chain?
4. RQ4: How can public blockchain confirmation latency be decoupled from student- and supervisor-facing operations without losing track of confirmation state?

1.7 Scope limitations and assumptions

The component covers the following academic event types: group registration, topic registration, TAF submission, TAF review, project proposal submission, approval, topic change request, checklist document submission and milestone document submission. It covers local DAG attestation, ledger verification, public anchoring on the Ethereum Sepolia testnet, background reconciliation, and the data behind the Integrity DAG and Audit Trail views. Duplicate-file detection and semantic similarity scoring are included as supporting checks, and the choice of embedding model is evaluated, but the development of new similarity algorithms is not. User interface components owned by other group members, and deployment on Ethereum mainnet, are outside the scope of this thesis.

The work has the following limitations. Sepolia is a public testnet; it reproduces real Ethereum transaction behaviour but not production cost, uptime or long-term governance. The prototype anchors group identifiers, reference identifiers and event

labels as readable strings for demonstration clarity, which a production deployment would need to pseudonymise. The evaluation concentrates on functional correctness and tamper detection. Apart from the similarity benchmark and individual timing observations, systematic performance measurement, such as gas cost and confirmation-time distributions, is left for future work.

The design assumes that the integrity service and its signing key are operated by the institution, that the owner wallet private key is protected in the service environment and never committed to source control, that the canonical snapshot for an event is deterministic for a given database state, and that Sepolia remains reachable through the configured remote procedure call (RPC) provider.

1.8 Individual contribution

The author was individually responsible for the research, design and implementation of the integrity governance component. This comprised the literature review and gap analysis; the group-wise DAG provenance model with its block, payload and edge structure; the canonical record snapshot hashing and two-level verification mechanism; the Solidity anchoring contract and its deployment and interaction layer using Web3.py and py-solc-x; the asynchronous anchoring flow, the separation of queue and anchor state, and the background and manual reconciliation mechanism; the integrity service endpoints consumed by the backend and the DAG visualisation; the duplicate-file and similarity checks hosted by the integrity service, including the comparison of three sentence-embedding models; the administrative Integrity DAG and Integrity Audit Trail views; and the six validation test cases, including tamper simulation. The design and implementation were also written up by the author as an IEEE-format research paper [16].

1.9 Thesis structure

Chapter 2 reviews the literature and synthesises the research gap and conceptual framework. Chapter 3 describes the research methodology, evaluation measures, ethical considerations and research artefacts. Chapter 4 presents the requirements, the architecture of the component within the complete solution, the design alternatives considered and the detailed design. Chapter 5 describes the implementation,

integration and testing. Chapter 6 reports the research, technical and integration results. Chapter 7 discusses the findings in relation to the literature and states the contributions, limitations and implications. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes the thesis and outlines future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

A structured literature review was carried out to inform the design of the component. IEEE Xplore was used as the primary database because it indexes the peer-reviewed conference and journal literature on blockchain engineering, distributed ledger performance and smart contract security. Searches were organised around the themes that the component depends on, using Boolean search strings such as "Ethereum" AND "Data Provenance", "Ethereum" AND "Performance Evaluation", "Ethereum" AND "security analysis" and "Public Blockchain" AND "Private Blockchain". A further group of studies on semantic similarity, plagiarism and novelty detection and document retrieval was added because the integrity service also performs similarity checks on submitted documents.

For each selected study, a structured extraction matrix recorded the publication year, technologies, tools, technology stack, methodology, problem addressed, results and performance, discussion and stated future work, identified gaps and the full citation. The extracted records were then grouped by theme and synthesised into comparative summaries and a gap analysis. In total, 36 records were extracted, as summarised in Table 2.1. One record on blockchain-based distributed knowledge graphs is not cited in this thesis because its complete bibliographic details could not be verified; the remaining 35 are cited where relevant, and the full extraction list is given in Appendix A.

Table 2.1: Review themes, search focus and number of extracted records

Theme	Search focus	Records extracted
Smart contract security and analysis	Ethereum smart contract security; "Ethereum" AND "security analysis"	10
Blockchain-based data provenance	"Ethereum" AND "Data Provenance"	7
Ethereum performance evaluation	"Ethereum" AND "Performance Evaluation"	4
Public versus private/permissioned platforms	"Public Blockchain" AND "Private Blockchain"	7

Theme	Search focus	Records extracted
Semantic similarity, novelty and document retrieval	Sentence embeddings, plagiarism and novelty detection, knowledge ownership, PDF retrieval	8
Total		36

The review concentrated on peer-reviewed literature published between 2017 and 2025. Grey literature, product white papers and non-peer-reviewed blog material were excluded from the evidence base, with the exception of a small number of foundational specifications that define the underlying technologies [1], [2], [17], [18].

2.2 Domain and theoretical foundations

Data provenance. Data provenance describes the origin of data and the processes and agents that produced or transformed it. The W3C PROV data model formalises provenance in terms of entities, activities and agents and the relations between them [17]. In research governance, provenance answers questions such as who submitted a topic, when the submission occurred, which supervisor reviewed it, whether the topic was later changed and which document was attached to a given stage. Because academic decisions depend on earlier states, an incomplete or mutable provenance trail weakens the ability to resolve disputes about authorship, supervision, originality and approval.

Cryptographic hashing and tamper evidence. A cryptographic hash function maps an input of arbitrary length to a fixed-length digest, such that a small change in the input produces a different digest. SHA-256, specified in the Secure Hash Standard, produces a 256-bit digest and is widely used for integrity checking [18]. Hashing alone does not prevent modification; it makes modification detectable when a trusted copy of the original digest is available. Linking each record's digest to the digest of its predecessor produces a hash chain in which altering any earlier element invalidates every later link.

Blockchain and smart contracts. A blockchain is an append-only ledger in which transactions are grouped into blocks that are cryptographically linked and replicated

across participants, with consensus rules determining which history is accepted [1]. Ethereum generalises this model with a virtual machine that executes smart contracts, programs whose code and state are stored on the chain [2]. Public blockchains such as Ethereum allow any external observer to verify that a transaction was included at a particular block and time. They are, however, not efficient general-purpose databases: on-chain storage is expensive and public, throughput is limited, and confirmation is slow compared with a conventional database [3], [4], [10]. Smart contracts are also difficult to change after deployment, so defects can have lasting consequences [19], [20].

Hash anchoring. Hash anchoring is a hybrid pattern in which a system computes the hash of an off-chain record or ledger state and submits only that hash to a blockchain. Anyone can later recompute the hash of the off-chain record and compare it with the on-chain value; a match shows consistency with the anchored state and a mismatch reveals tampering or inconsistency. This pattern underlies most of the hybrid provenance systems reviewed in Section 2.3 [6], [7], [11], [13].

Directed acyclic graphs. In a linear blockchain, every block references exactly one predecessor. A DAG generalises this structure by allowing nodes to represent events and edges to represent dependencies without forcing every event to share the same immediate predecessor. DAG structures have been used in blockchain systems to model the genealogy of correlated records [6] and to track parallel branches of a chain [21]. For a workflow in which many research groups progress independently, a DAG can express group-level structure that a single global sequence would hide.

Public, permissioned and hybrid platforms. Public platforms such as Ethereum provide open verifiability, whereas permissioned platforms such as Hyperledger Fabric restrict participation to known organisations and generally achieve lower latency and higher throughput [3], [22], [23]. Consensus mechanisms strongly influence performance: proof-of-authority (PoA) propagates blocks faster than proof-of-work (PoW) in private Ethereum networks [24], and proof-of-stake (PoS) reduced transaction latency relative to PoW in a cloud provenance setting [13]. Hybrid architectures that combine a public chain with a permissioned one, or a

blockchain with off-chain storage, attempt to balance verifiability, privacy and performance [12], [14].

2.3 Related research

Blockchain-based provenance in healthcare and the IoMT. Healthcare is the most common domain for blockchain provenance research because medical records require integrity, privacy, traceability and controlled access. Sun et al. proposed a provenance-aware e-healthcare system that combines Ethereum, IPFS and smart contracts with a DAG-oriented data model for the genealogy of correlated electronic health records; their zero-in-degree storage strategy reduced query times compared with linear methods, but resource demand and gas cost increased as the number of linked record nodes grew [6]. BESURE combined Ethereum, encrypted cloud storage and a server-aided password-hardening protocol to provide secure provenance for electronic health records; validating a provenance record took around 2.8 minutes, and the gas cost of deploying and invoking contracts had to be borne by users [7]. D'Antonio and Uccello proposed a three-layer provenance architecture for IoT-based healthcare in which tamper-proof storage improved trust in data, but they noted high transaction cost, the energy inefficiency of PoW and a conflict between blockchain immutability and the right to erasure under data protection regulation [8].

Zhuang et al. developed a patient-centric health information exchange framework in which smart contracts give patients control of access to their records; a simulation of 1,553,635 transactions achieved a 100% success rate, but the authors identified Ethereum's intrinsic throughput of about 13–15 transactions per second as a scalability constraint [10]. Rehman et al. proposed a hybrid IoMT model that bridges Ethereum and Hyperledger Fabric, analysed with an M/M/1 queuing model whose theoretical latency of 0.625 s closely matched a Monte Carlo simulation, while noting the added complexity and centralisation risk of private chains [12]. Taken together, these studies support keeping sensitive data off-chain and using the chain for verifiable proofs, but none of them addresses academic lifecycle events such as topic revision, supervisor approval, group branching or milestone document governance.

Blockchain for cloud storage integrity and data governance. Zhang et al. used a public blockchain to make cloud storage auditing resistant to procrastinating auditors, achieving constant communication overhead and allowing a user to audit the auditor's behaviour for 50 entries in under one minute, but they identified PoW energy consumption as a drawback [9]. Patil et al. combined AWS S3, IPFS and Ethereum to provide provenance assurance for a university cloud storage application; read latency remained around 0.03 s, but add latency grew with load and network size, from 3.48 s to 45.9 s when the network increased from one to three nodes, and 20 transactions took approximately 417.9 s [11]. Abhishek et al. proposed a decentralised cloud provenance mechanism using Ethereum and IPFS with an optimised PoS consensus and showed lower transaction latency than PoW across the tested scenarios [13]. Garcia et al. combined blockchain, proxy re-encryption and BBS signatures for consent-driven, privacy-preserving data governance in multi-stakeholder applications; re-encryption operations completed in under 50 ms, and Byzantine-fault-tolerant platforms produced block times of about 5 s compared with 10–32 s on an Ethereum test network [14].

These works support two design decisions adopted in this thesis. First, the blockchain should not store full documents or private metadata. Second, an interactive workflow should not wait synchronously for blockchain finality, because confirmation delay grows with load and network conditions.

Blockchain platform performance. Performance studies consistently show that platform and consensus choice affect latency and throughput. Pongnumkul et al. found that Hyperledger Fabric completed 10,000 transactions in 62.59 s compared with 485.41 s for Ethereum, although Ethereum handled more concurrent transactions (50,000 compared with 20,000) [3]. Dabbagh et al. and Abhishek et al. likewise reported that Fabric outperformed Ethereum on their measured metrics, with the latter attributing Ethereum's higher latency to PoW and observing larger average Fabric block sizes (256 kB compared with 80 kB) [23], [25]. Monrat et al. found that Fabric outperformed Ethereum and Quorum by a large margin and Corda slightly [22].

At a finer level, Toyoda et al. traced the throughput limits of private PoA Ethereum to signature recovery (Ecrecover), with median throughput of 1,250 transactions per second for simple contracts falling to 40 for CPU-intensive workloads [26]. Chen et al. showed that transaction-oriented and block-oriented latency both grow with the number of network hops, and that PoA propagates blocks faster than PoW [24]. Choi and Hong benchmarked a private Ethereum network against the Ropsten testnet with Hyperledger Caliper and found higher latency (about 14 s compared with 4.5 s) and lower throughput on the testnet, whose results were volatile because of its small number of miners [4]. Saingre et al. proposed BCTMark for reproducible blockchain benchmarking and measured, for example, a 44% power increase for Ethereum Clique on Raspberry Pi devices under load [27], while Horvat et al. reported a carbon footprint per transaction about 100 times smaller for a private Corda energy-trading platform than for a comparable Ethereum transaction [15]. For this thesis, the most important implication is that public testnet confirmation is slow and unpredictable, so it must not lie on the critical path of user requests.

Smart contract security. Because deployed contracts are difficult to correct, their security has received substantial attention. Systematic reviews by Kushwaha et al. catalogued Ethereum analysis tools, finding that static analysis dominates and that Slither, Mythril and Oyente performed best for vulnerability detection [28], and classified vulnerabilities into three root causes (the Solidity language, Ethereum virtual machine features and blockchain design) with 17 sub-causes, examining 24 vulnerabilities in depth [19]. Dika and Nowostawski found SmartCheck to be the most effective and Oyente the most accurate of the tools they evaluated, but observed significant inconsistencies between tools [20]. Ji et al. evaluated countermeasures on 237 labelled contracts and showed that tools with high detection rates often suffer from many false positives [29].

Other studies propose specific assurance techniques. MuSC provides mutation testing with Solidity-specific mutation operators [30]; Zhou et al. combined syntax-topology analysis and symbolic execution to locate logic risks, analysing 2,952 contracts at an average of 612 s each [31]; EtherGIS used graph neural networks over control-flow graphs and reached accuracy above 95%, although with

high false negatives for re-entrancy [32]; and HoRStify offered provably sound static analysis with a precision loss of only 0.5–5.4% relative to Securify at about 2.3 times the execution time [33]. Huang et al. studied upgradeable contracts, detecting 127,753 upgradeable contracts in the first 12 million Ethereum blocks, and warned that upgrade mechanisms let a small group of owners change contract logic after deployment [5]. Frauenthaler et al. showed that on-chain verification can be extremely gas-intensive, with validation of a single Ethereum block header costing around three million gas, and reduced relay cost by up to 92% through validation-on-demand [21]. These findings motivate a contract that is intentionally minimal, non-upgradeable and inexpensive to call.

Similarity, novelty and knowledge ownership. Research integrity also involves originality. Sentence-embedding approaches compare documents semantically rather than lexically: Bacciu and Carta incorporated word order into sentence embeddings using a linear autoencoder for sequences [34], and Ha et al. showed that SBERT reduced the time to find the most similar question from 795 s with BERT to about 0.828 s with only a slight loss of accuracy [35]. Agrawal et al. combined SBERT with transformer layers for author-oriented semantic plagiarism detection and outperformed a CNN baseline (F1 of 0.75 compared with 0.72) [36], while Chikkamath et al. found that a classical NBSVM model was more accurate and about 12 times faster than deep models for patent novelty detection, and that BERT-family models performed poorly on domain-specific patent language [37]. Wang et al. used blockchain to record the evolution of a decentralised knowledge graph with multiple contributors, reaching about 96% accuracy with 100 contributors [38], illustrating how ledgers can support contribution tracking and ownership in collaborative knowledge work.

2.4 Existing systems products and techniques

Conventional research management and learning management systems record workflow history through database timestamps, audit columns and application logs. These techniques are inexpensive and easy to query, but their trustworthiness depends entirely on the operator: an administrator with write access to both the data

and the log can alter history without detection. They therefore provide accountability only to the extent that the institution's infrastructure is trusted.

Several technique families from the literature are directly relevant to the component. **Off-chain storage with on-chain hashes** is used in provenance systems built on IPFS, cloud object storage or relational databases together with Ethereum [6], [7], [11], [13]. **DAG-oriented provenance** is used to represent record genealogy [6] and branch structure [21]. **Benchmarking frameworks** such as Hyperledger Caliper and BCTMark support reproducible measurement of latency and throughput [4], [27]. **Smart contract analysis tools** such as Slither, Mythril, Oyente, Securify and SmartCheck support pre-deployment security review, with known trade-offs between detection rate and false positives [20], [28], [29]. **Document intelligence techniques** support similarity checking and retrieval: sentence embeddings for semantic similarity [34], [35], transformer-based plagiarism detection [36], and vector search with FAISS combined with large language models for question answering over PDF documents, which reported response times of about 1–2 s for 14-page PDFs and 1.9 s in a multi-document setting [39], [40].

What is missing from these existing techniques is an integrated, domain-specific mechanism that ties each academic workflow event to a verifiable proof, organises those proofs according to the structure of the research lifecycle, and remains usable in an interactive web application.

2.5 Gap synthesis and conceptual framework

Table 2.2 synthesises the gaps identified in the review and the response adopted by this component. The gaps were first formulated during the review stage of the project and were then refined as the design matured.

Table 2.2: Gap synthesis and corresponding design response

Gap	Evidence from existing work	What is missing	Response in this component
G1: No blockchain model of academic governance workflows	Systems focus on medical records, cloud files, IoT data and supply-chain style tracking [6]–[8], [11]	A formal model of the research lifecycle: group registration, topics, TAFs, proposals, approvals, topic changes and milestones	Nine typed academic events with reference identifiers and group scope
G2: Weak event-level accountability	Systems store file hashes or metadata and track operations [11], [13]	Proof of which event a record belonged to, which group submitted it and which approval changed the effective topic	Canonical snapshot of the event record, hashed and stored in the ledger block
G3: Linear chains hide multi-group structure	A single chain orders all transactions globally [1]	Representation of independently progressing groups and their relation to an overall timeline	Group-wise DAG branches connected to a main checkpoint chain
G4: Tamper detection asserted rather than tested	Immutability is assumed to guarantee integrity	Verification of live database state against stored proofs, and simulated tampering	Two-level verification (block hash and record hash) and tamper scenarios
G5: Confirmation latency treated as acceptable	Ethereum and testnets show slow, volatile confirmation [3], [4], [11]	A user-facing workflow that does not wait for public confirmation	Synchronous local attestation with asynchronous anchoring and reconciliation
G6: Privacy, cost and rigidity of on-chain logic	Gas cost, regulatory conflict and contract immutability risks [5], [7], [8]	Minimal on-chain footprint with flexible off-chain workflow logic	Off-chain data and workflow; minimal, non-upgradeable anchoring contract

The review also identified two further gaps that are acknowledged but not addressed by the implemented component. Existing tools do not capture the evolution of contributor roles over time, for example a change in the academic role that a person holds within a project, and they do not apply anomaly detection to flag unusual revision or approval behaviour. These are discussed as future directions in Chapter 9.

Figure 2.1 presents the resulting conceptual framework. Each gap maps to a design response, and the responses jointly target the intended outcome: tamper-evident and publicly verifiable academic provenance that neither blocks users nor exposes private records. The framework rests on a principle that recurs throughout the literature: each technology should be used where it is strongest. A relational database stores structured academic data efficiently, a local hash-linked ledger provides fast provenance and tamper detection, and a public blockchain provides an external timestamp and proof that the institution cannot silently rewrite.

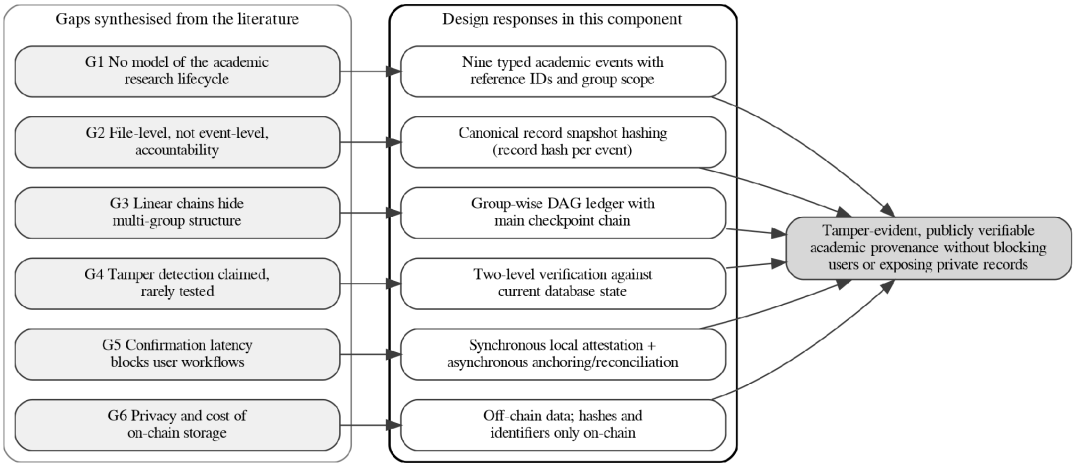


Figure 2.1: Conceptual framework linking the identified gaps to the design responses of the component

3 RESEARCH METHODOLOGY

3.1 Research design

The research follows a design-oriented, constructive approach in which a technical artefact is designed to address an identified problem and is then evaluated against that problem. This approach suits the research questions, which ask *how* integrity guarantees can be achieved under specific constraints of privacy and usability, and which are answered by demonstrating a working design and examining its behaviour. The research was conducted in six phases.

1. **Problem identification and literature review.** The governance problem was characterised and the literature reviewed using the method described in Section 2.1, producing the gap synthesis in Table 2.2.
2. **Requirement derivation.** Functional and non-functional requirements were derived from the gaps, from the platform workflow and from the limitations reported in the literature (Section 4.1).
3. **Design and evaluation of alternatives.** Candidate designs were compared, including a purely local blockchain-inspired ledger, a fully on-chain Ethereum application and hybrid variants (Section 4.3).
4. **Iterative implementation.** The component was implemented and integrated with the platform in increments, starting with local attestation and verification and then adding public anchoring and reconciliation.
5. **Validation.** The deployed component was exercised through six test cases: an end-to-end success flow, duplicate-file rejection, semantic similarity checking, and three kinds of deliberate tampering (Sections 3.3 and 6.2).
6. **Reflection and refinement.** Observations from validation were fed back into the design. The most significant refinement was the replacement of synchronous receipt waiting by asynchronous anchoring with reconciliation, after synchronous waiting was observed to freeze the user interface (Section 6.2).

The design itself evolved during the project. The initial plan, recorded in the project groundwork, was a Python-based, blockchain-inspired ledger using JSON or SQLite storage and hashlib, with Ethereum anchoring planned as a later extension. This plan offered full control, easy tampering simulation and fast bulk experimentation, which

suited research evaluation. As the platform matured, the ledger was re-implemented within a FastAPI service backed by PostgreSQL, and real anchoring on Ethereum Sepolia was added so that the integrity guarantee no longer depended solely on the institution's own server. The final design therefore retains the controllability of a local ledger and adds external verifiability.

3.2 Data participants or study materials

No human participants took part in the study, and no interviews or surveys were conducted. Three kinds of material were used.

- **Literature corpus.** The 36 records extracted from IEEE Xplore (Section 2.1 and Appendix A) informed the requirements, the design decisions and the comparison in Chapter 7.
- **Platform test data.** Test accounts (for example, a test student account) were used to create research groups with codes such as R26-SE-001 to R26-SE-019, together with their topics, TAFs, proposals, review decisions, topic change requests, checklist documents and component submissions. All tampering was carried out on these test records, never on genuine academic records.
- **Similarity corpus.** To choose an embedding model, 22 documents from the project's development submissions folder were compared pairwise (231 pairs). They included proposal reports, TAFs, project and research papers, and a small number of generated test TAFs. The text of each document was capped at 20,000 characters. The documents were processed locally, and only aggregate scores are reported.

Anchoring used the public Ethereum Sepolia testnet (chain ID 11155111), which behaves like Ethereum but carries no financial cost. Transactions were signed by a dedicated MetaMask test wallet funded with test Ether and were submitted through an Alchemy RPC endpoint.

3.3 Experimental or study procedure

Validation was organised as six test cases, each with a stated purpose. Every test used the deployed services and their normal interfaces, so the whole integration path was exercised.

1. **TC1, success flow.** A group was registered through the student interface. The test then checked that the group row existed in the database, that a group-registration block and its main checkpoint had been added to the ledger, that the anchor row had reached the attested state with a Sepolia transaction hash and block number, that the public verify endpoint reported a valid ledger, and that the event appeared as On-chain in the Integrity DAG and Audit Trail. *Purpose: prove that the block exists locally and is anchored externally.*
2. **TC2, duplicate-file prevention.** A proposal PDF that had already been submitted was uploaded again. *Purpose: show that document integrity goes beyond status tracking.*
3. **TC3, semantic similarity.** Three sentence-embedding models were first compared on the 22-document corpus. A TAF was then submitted for group R26-SE-011 and its similarity score and decision were recorded. *Purpose: show that checking is not limited to exact hash matching.*
4. **TC4, academic record tampering.** The supervisor of group R26-SE-002 was changed with a direct SQL update on the `research_groups` table, bypassing the application. The Integrity DAG and Audit Trail were then inspected. *Purpose: show that tampering with academic data is detected.*
5. **TC5, ledger block tampering.** One character of the stored hash of ledger block 45 was changed with a direct SQL update on the `blocks` table. *Purpose: show that the local ledger behaves as immutable, because any edit is exposed.*
6. **TC6, anchor link tampering.** One character of the `block_hash` stored in the `blockchain_anchors` row for R26-SE-004's TAF submission was changed. *Purpose: show that tampering with the evidence link is detected.*

Before each tampering test, the original value was saved in a comment beside the SQL statement, so that the record could be restored afterwards.

3.4 Measures and evaluation metrics

Most measures are observable outcomes: whether a block exists, which status the verifier reports, and how the administrative views label an event. Table 3.1 links each measure to the research question it informs.

Table 3.1: Evaluation measures and their relation to the research questions

Measure	Definition	Type	RQ
Branch placement	Each academic event block, and the checkpoint that follows it, carries the correct group and branch data	Functional	RQ1
Ledger validity	The verify endpoint reports is_valid, local_valid and blockchain_valid, with a list of invalid blocks	Functional	RQ1, RQ2
Record-level tamper detection	A direct database edit to an attested record makes its block Invalid	Functional	RQ2
Block-level tamper detection	An edited block hash makes that block, and the block that links to it, Invalid	Functional	RQ2
Evidence-link tamper detection	An edited anchor link makes the event lose its on-chain status	Functional	RQ3
Data minimisation on-chain	Anchors carry only hashes, identifiers, event type, timestamp and wallet	Design inspection	RQ3
Non-blocking user path	The user request completes without waiting for a transaction receipt	Functional	RQ4
Registration-to-confirmation time	Time from creation of a record to its anchor row being marked attested	Observed timing	RQ4
Duplicate-file rejection	A file identical to an earlier submission is refused	Functional	Supporting
Similarity separation	For each model: pairs above the 0.8 threshold, maximum and mean pairwise score	Quantitative	Supporting
Model cost	For each model: load time and encoding time for the corpus	Quantitative	Supporting

3.5 Data analysis

For each test case, the observed outcome was compared with the outcome the design predicts. Evidence was gathered from three independent places where possible: the database itself, through SQL queries on the `research_groups`, `blocks` and `blockchain_anchors` tables; the public verify endpoint; and the administrative views. Using several sources guards against trusting a single view of the system under test.

For the similarity benchmark, each model encoded the same 22 documents and every pair was scored by cosine similarity. For each model, the number of pairs at or above

0.8, the maximum and mean scores, and the load and encoding times were recorded. A useful model should place genuinely related documents well above the general level of similarity between unrelated documents. A low mean score with high-scoring true matches therefore indicates good separation. Timing was read from a single run, so it shows relative cost rather than a stable benchmark.

3.6 Ethics privacy security and responsible AI

Ethics and data. No human participants were involved. Every workflow and tampering test used test accounts and test groups, and records were restored afterwards. The similarity benchmark used real academic documents from the project's submissions folder. They were processed locally, never published or sent to third parties, and are reported only as aggregate statistics, without file names or author identifiers.

Privacy by design. The architecture deliberately keeps student and supervisor identities, group membership, topic details, submission metadata, file paths, review notes, extracted document text and the local DAG blocks off-chain. Only hashes, identifiers, the event type, the blockchain timestamp and the anchoring wallet address are placed on-chain. This follows the principle that data written to a public blockchain cannot practically be removed, which conflicts with the right to erasure under data protection regulation [8], and aligns the design with data protection obligations such as those of Sri Lanka's Personal Data Protection Act [41]. The prototype stores group and reference identifiers as readable strings for demonstration clarity; because even metadata can reveal information when identifiers are meaningful, a production deployment should hash or pseudonymise these identifiers before anchoring.

Security. Only the owner wallet can anchor blocks, which prevents arbitrary users from inserting misleading anchor records. The owner's private key is held in the integrity service environment configuration and must never be committed to source control. The contract is intentionally minimal and non-upgradeable, in line with evidence that complex or upgradeable contracts increase the attack surface and centralisation risk [5], [19].

Responsible use of automated analysis. The similarity analysis hosted by the integrity service produces a similarity score and a decision status, such as accepted or needs review. It is designed as decision support for supervisors rather than as an automatic finding of misconduct, and submissions from the same group or user are excluded from matching to avoid flagging legitimate resubmissions. The use of generative AI tools in preparing this thesis is disclosed in Appendix B.

3.7 Reproducibility and research artefacts

The research produced the following artefacts:

- the integrity service source code (FastAPI), covering snapshot reconstruction, the ledger, verification, the DAG, duplicate and similarity checks, and blockchain anchoring;
- the Solidity anchoring contract (compiled with Solidity 0.8.20 through py-solc-x 2.0.4), its deployment script and the generated ABI artefact;
- the backend integration code for the attestation queue and its worker;
- the Docker deployment configuration, whose dependency file (`requirements-docker.txt`) pins the library versions listed in Table 5.1;
- the similarity benchmark script (`scripts/compare_documents.py`), which writes ranked document pairs and a model summary to the `results` folder;
- the six test cases, with their SQL statements, saved original values and screenshots (Section 6.2).

The source code is held in a private GitHub repository and can be made available to examiners on request. The deployed integrity service is reachable at integrity.researchlens.tech, and every anchor can be checked independently on Sepolia using the transaction hashes recorded in the `blockchain_anchors` table.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the research gaps (Table 2.2), from the platform's academic workflow and from the limitations reported in the literature. Table 4.1 lists the functional requirements and Table 4.2 the non-functional requirements.

Table 4.1: Functional requirements of the integrity governance component

ID	Requirement	Source
FR1	Attest each of the nine academic event types when the backend records the event, identified by event type and reference identifier	G1, G2
FR2	Reconstruct a canonical snapshot of the fields relevant to the event and compute its SHA-256 record hash	G2
FR3	Append a hash-linked block to the correct group branch and connect the branch to the main checkpoint chain	G3
FR4	Verify every block hash, every previous-hash reference and every academic event record hash against the current database state	G4
FR5	Anchor the block hash and record hash of each academic event block on Ethereum Sepolia, without documents or private content	G6
FR6	Prevent duplicate anchoring of the same block hash, both in the contract and before broadcasting	G6
FR7	Record anchor state (pending, attested, failed) with transaction hash, block number, contract address and chain identifier	G5
FR8	Reconcile pending anchors automatically in the background, and on demand through an API endpoint	G5
FR9	Provide DAG data for an overall view and a selected-group view, with a proof-state label for every node	G3
FR10	Provide similarity analysis for TAF, proposal and selected milestone submissions, excluding same-group and same-user matches	Platform workflow
FR11	Reject a document whose file hash matches an earlier submission	Platform workflow
FR12	Provide an administrator audit trail showing local and Sepolia status per event, with search, filters, CSV export and a copyable proof	G2, G4

Table 4.2: Non-functional requirements of the integrity governance component

ID	Quality attribute	Requirement
NFR1	Responsiveness	User-facing operations must not wait for public blockchain confirmation
NFR2	Privacy	No document, document text, review note or personal detail may be written on-chain
NFR3	Tamper evidence	Any post-attestation change to a ledger block or an attested record must be detectable
NFR4	Reliability	Failed integrity handoffs must be retried, and duplicate processing of the same event must be prevented
NFR5	Security	Anchoring must be restricted to the owner wallet; secrets must be held outside source control
NFR6	Maintainability	Workflow rules must remain off-chain so they can change without redeploying contracts
NFR7	Separation of concerns	Blockchain logic, wallet configuration and gas handling must be confined to the integrity service
NFR8	Access control	Supervisors must only be able to inspect the branches of groups they supervise

4.2 Component architecture within the complete solution

The complete solution has five layers, shown in Figure 4.1. The **frontend** (React and Vite) provides student, supervisor and administrator interfaces, including the Integrity DAG and Integrity Audit Trail in the administrator workspace. The **backend workflow service** (Node.js and Express) handles authentication, workflow validation and document upload, saves academic records and enqueues integrity attestation events. The **storage layer** consists of PostgreSQL, which holds groups, topics, submissions, reviews, checklist requirements and the attestation queue, and backend file storage for uploaded documents. The **integrity governance module** (FastAPI), which is the individual component of this thesis, reconstructs record snapshots, computes hashes, maintains and verifies the local DAG ledger, performs duplicate and similarity checks on documents, and anchors block hashes. The **anchoring layer** consists of a Solidity contract on Ethereum Sepolia, accessed with Web3.py through an Alchemy RPC endpoint.

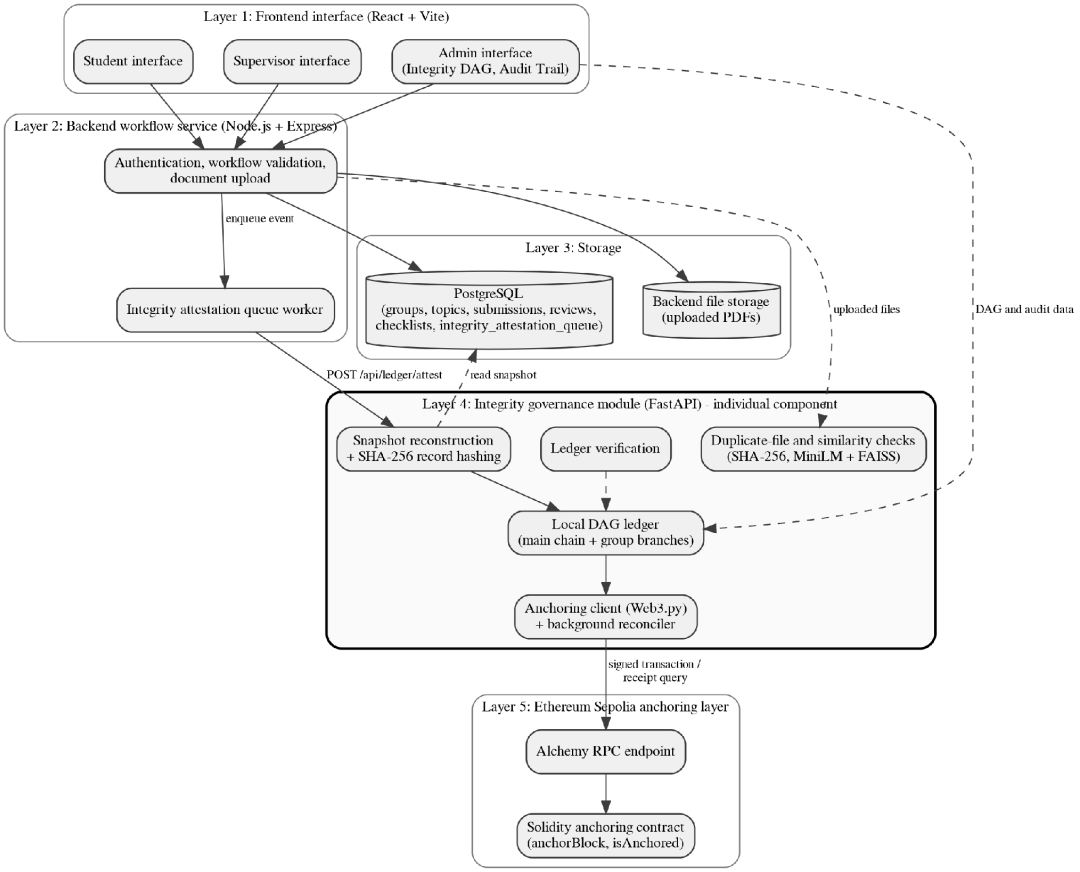


Figure 4.1: Layered architecture of the complete solution, with the integrity governance component highlighted

A central architectural decision is the separation of private operational data from public proof. Table 4.3 summarises where each kind of data is stored. This placement follows the hybrid pattern observed in healthcare and cloud provenance systems [6], [7], [9], [11], [13], while ensuring that no academic content is exposed publicly.

Table 4.3: Placement of data between off-chain storage and the blockchain

Off-chain (database, file storage, integrity service)	On-chain (anchoring contract on Sepolia)
Student and supervisor identities; group membership	Local block hash
Research topic details and submission metadata	Record hash
Uploaded PDF paths and file hashes	Event type
Supervisor review notes; checklist configuration	Reference identifier

Off-chain (database, file storage, integrity service)	On-chain (anchoring contract on Sepolia)
Local DAG blocks	Group identifier
Extracted document text for similarity checking	Blockchain-generated timestamp; anchoring wallet address

4.3 Design alternatives and decisions

Several alternatives were considered. The first decision concerned the overall integrity mechanism. Table 4.4 compares a local, blockchain-inspired ledger with a fully on-chain Ethereum application, based on the comparison prepared during the groundwork phase of the project.

Table 4.4: Comparison of a local blockchain-inspired ledger with a fully on-chain Ethereum application

Aspect	Local hash-linked ledger	Ethereum and Solidity application
Type of system	Blockchain-inspired, institution-operated ledger	Decentralised application on a public chain
Workflow flexibility	Full control over dynamic workflow logic	Smart contracts are rigid and costly to change
Frequent updates	Handled locally at database speed	Each change requires a transaction with latency and cost
Data volume and storage	Full records and documents stored efficiently	On-chain storage is costly and not scalable
Query and analysis	Rich querying of revisions, timelines and contributions	Limited querying and analytics
Tampering simulation	Possible, enabling verification experiments	Not possible on the chain itself (immutable by design)
Independent trust	None beyond the institution's own server	Public, externally verifiable proof

Neither option satisfies all requirements alone. A local ledger can detect tampering only if the verifier trusts the server that stores it; an attacker who modifies both the database and the local ledger could rewrite history undetected. A fully on-chain workflow would be slow, expensive and difficult to change, and it would expose academic data. The adopted design is therefore **hybrid**: dynamic workflow logic and full data remain off-chain, a local hash-linked ledger provides immediate provenance

and tamper detection, and the public chain stores only proofs that the institution cannot silently rewrite.

The remaining decisions were as follows.

- **DAG instead of a single linear chain.** A single linear ledger is simple but interleaves the events of all groups and obscures each group's progress. A DAG with a main checkpoint chain and one branch per group preserves the overall timeline while supporting group filtering and supervisor-specific views (G3).
- **Asynchronous instead of synchronous anchoring.** Waiting for the transaction receipt inside the request was implemented first and was observed to freeze the interface, sometimes for more than 120 seconds. Literature confirms that Ethereum and testnet confirmation is slow and volatile [3], [4]. The final design broadcasts the transaction, records it as pending and confirms it later (NFR1).
- **Public Ethereum instead of a permissioned platform.** Permissioned platforms such as Hyperledger Fabric achieve better latency and throughput [3], [22], [23] but require consortium governance and reduce public verifiability. Because the aim is evidence that is independent of the institution, a public Ethereum network was preferred, with Sepolia used for development and demonstration. The same contract pattern could later be deployed to Ethereum mainnet, a layer-2 network or a permissioned chain, depending on cost, privacy and governance requirements.
- **Anchoring academic event blocks only.** Main checkpoint blocks organise the DAG structurally but carry no academic content, whereas academic event blocks contain the meaningful proofs. Only academic event blocks are therefore anchored, which reduces transaction volume and cost.
- **Minimal, non-upgradeable contract.** Upgradeable proxy patterns would allow logic to change after deployment but introduce centralisation and trust risks [5]. Because the anchoring logic is simple and stable, a non-upgradeable contract was chosen; if new functionality is needed, a new contract can be deployed and its address recorded in configuration.

- **Blockchain logic confined to the integrity service.** The backend never creates blockchain transactions. Controllers therefore do not need to know contract details, wallet configuration, gas parameters or transaction receipts (NFR7).

4.4 Detailed design

Academic events. The component attests the nine event types listed in Table 4.5. Each event carries a reference identifier that points to the corresponding database record, for example the group identifier for group registration and the proposal row identifier for proposal submission. The module reconstructs the snapshot for the given event and reference identifier before hashing it.

Table 4.5: Academic event types attested by the component

Event type	Lifecycle meaning
group_registration	A research group is registered
topic_registration	A research topic is registered for a group
taf_submission	A topic assessment form is submitted
taf_review	A supervisor reviews a topic assessment form
project_proposal	A project proposal is submitted
approval	A supervisor approval decision is recorded
topic_change_request	A change to the registered topic is requested
checklist_document_submission	A document required by a stage checklist is submitted
milestone_document_submission	A milestone deliverable is submitted

Block structure and hashing. Each block is stored as a row of the `blocks` table with the columns `idx`, `timestamp`, `data`, `previous_hash` and `hash`. The block hash is computed over the index, timestamp, data and previous hash, so changing any of them, or the link to the previous block, invalidates the block. The `data` column holds a JSON payload. Its keys, as observed in the deployed ledger, are listed in Table 4.6.

Table 4.6: Payload keys of academic event blocks and main checkpoint blocks

Key	Academic event block	Main checkpoint block
event	Event type, for example group_registration	main_checkpoint

Key	Academic event block	Main checkpoint block
ref_id	Identifier of the database record	Not used
record_hash	SHA-256 hash of the canonical record snapshot	Not used
chain_scope	group	main
group_id	Research group, for example R26-SE-011	Group whose branch was extended
branch_id	The group's branch	main
group_head_hash	Not used	Hash of the group branch head that the checkpoint ties into the main chain

In the deployed ledger, every academic event block is immediately followed by a main checkpoint block. For example, the group registration of R26-SE-011 was written as block 121, and block 122 is a main checkpoint whose `group_head_hash` equals the hash of block 121 (Section 6.2). This is also why the event numbers in the audit trail advance in steps of two.

Record snapshot hashing. For each event, the module reconstructs a canonical database snapshot that includes only the fields relevant to that event, and computes its SHA-256 hash. The record hash is stored inside the block payload. Two independent levels of verification result. *Block hash verification* detects changes to the ledger itself, and *record hash verification* detects changes to the underlying database record. The second level is what allows the component to detect tampering performed directly in the database, which conventional audit logs cannot.

DAG structure and edges. The ledger contains two kinds of nodes. *Academic record blocks* belong to group branches and represent real workflow events; these blocks are anchored on Sepolia. *Structural checkpoint blocks* belong to the main chain, starting from a genesis block, and connect group branch heads to the overall ledger timeline; they are local structural nodes and are not individually anchored. The DAG contains three edge types: *previous* edges connect consecutive blocks within the same branch or chain, *checkpoint* edges connect the main chain to group branch heads, and *anchor* edges visually represent the relationship between a group

event and its main checkpoint. Figure 4.2 illustrates the structure with two groups; as described above, each academic event produces one checkpoint on the main chain.

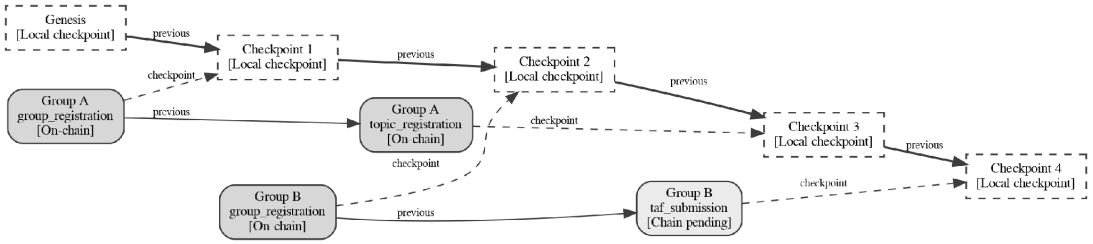


Figure 4.2: Illustrative DAG ledger with a main checkpoint chain and two group branches (node labels show proof state)

Views and proof-state labels. The DAG can be viewed as a whole, showing the main checkpoint chain and all group branches, or filtered to a single group. Each node shows the first characters of its block hash and one of the proof-state labels in Table 4.7. Main structural nodes are deliberately labelled "Local checkpoint" rather than "Not anchored", so that the hybrid design is not mistaken for a failure to anchor. A node whose record no longer verifies is drawn with a red border and marked Invalid.

Table 4.7: Node proof-state labels in the DAG visualisation

Label	Meaning
Local checkpoint	Genesis and main checkpoint nodes; structural and local by design
On-chain	Academic record block whose anchor is confirmed on Sepolia
Chain pending	Anchor transaction broadcast but not yet confirmed
Chain failed	Anchor transaction failed
Not anchored	Blockchain enabled, but no valid anchor is linked to the academic block
Invalid	The block or its record failed local verification

The Integrity Audit Trail lists one row per academic event, with separate Local and Sepolia columns, the Sepolia block number, the submission time, a link to the transaction and a "Copy proof" action. It can be searched by group, reference, hash or transaction, filtered by event type and status, and exported as CSV. Reading the two columns together gives the combined interpretations in Table 4.8.

Table 4.8: Interpretation of combined local and Sepolia status

Local	Sepolia	Meaning
Valid	On-chain	The current record matches the ledger and public proof exists on Sepolia. This is the expected state.
Valid	Queued or pending	The ledger is valid, but anchoring is still in progress.
Valid	Missing on-chain	The ledger is valid, but the public proof is missing or the contract address is wrong.
Invalid	On-chain	The record was changed after anchoring. Sepolia proves that the original hash existed.
Invalid	Missing on-chain	Ledger validation failed and no public proof is available for this block.
Valid or Invalid	Chain check retrying	The live Sepolia check failed temporarily, usually because of RPC or network delay. This does not by itself indicate tampering.
Valid	Chain failed	The record is valid, but the anchoring transaction failed and needs retry or investigation.

Anchoring contract. The Solidity contract acts as a public notary for local block hashes. It does not manage the workflow or store documents. Each anchor record contains the fields in Table 4.9. The contract exposes one primary state-changing function, `anchorBlock(bytes32 blockHash, bytes32 recordHash, string eventType, string refId, string groupId)`, which verifies that the block hash is non-zero and has not already been anchored, stores the anchor record and emits an event. A read-only `isAnchored` function reports whether a block hash has already been anchored. Anchoring is restricted to the owner address. The contract implements no workflow logic, file management, payments, upgradeability or external contract calls.

Table 4.9: Fields of an on-chain anchor record

Field	Content
blockHash	Hash of the local ledger block (bytes32)
recordHash	Hash of the canonical record snapshot (bytes32)
eventType	Academic event type
refId	Reference identifier of the database record
groupId	Research group identifier

Field	Content
anchoredAt	Timestamp generated by the blockchain
anchoredBy	Wallet address that anchored the block
exists	Flag used to reject duplicate anchoring

Attestation flow. Figure 4.3 shows the final attestation flow, which separates immediate local attestation from delayed public confirmation. The backend saves the academic record, inserts or updates an integrity queue item, and its worker calls the attestation endpoint. The integrity module reconstructs the snapshot, computes the record hash, creates the local DAG block, broadcasts the Sepolia transaction, stores a pending anchor row with the transaction hash and responds immediately. The backend then marks the queue item as attested, and the background reconciler later confirms the transaction.

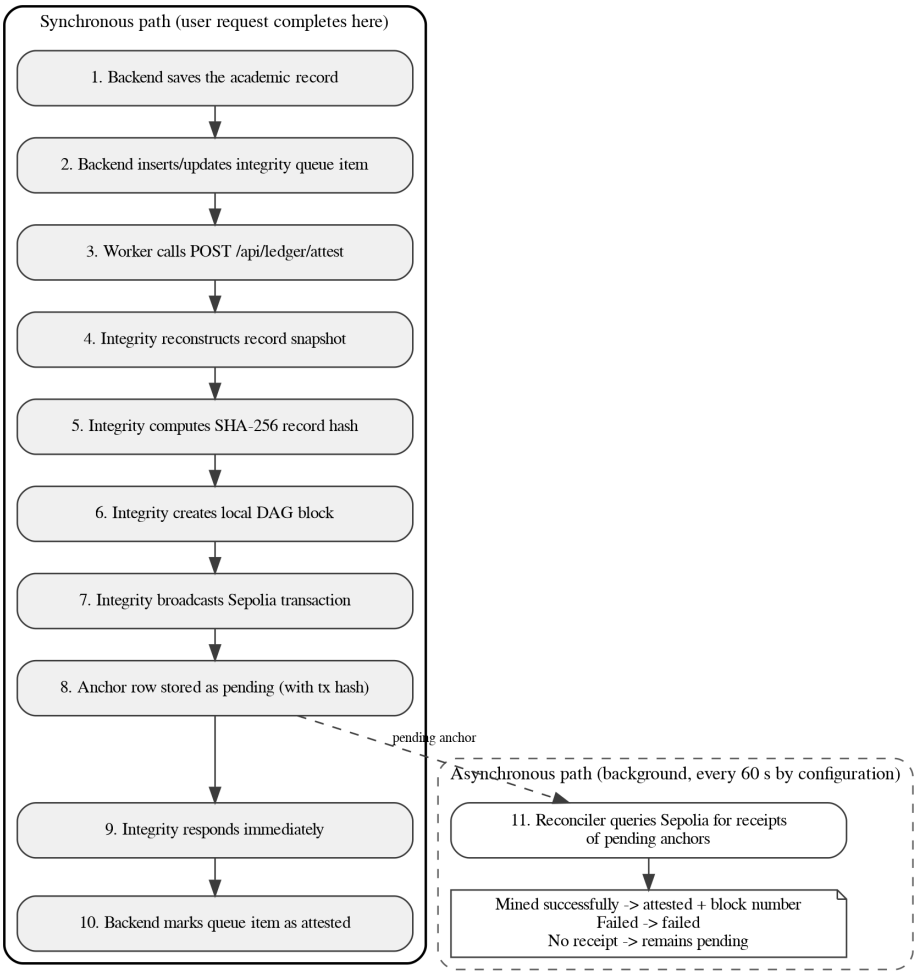


Figure 4.3: Attestation flow separating synchronous local attestation from asynchronous blockchain confirmation

Queue and anchor state. Two tables with distinct responsibilities hold the state of an attestation, as shown in Figure 4.4. The `integrity_attestation_queue` table belongs to the backend; its status shows whether the backend has handed the event to the integrity module. The `blockchain_anchors` table belongs to the integrity module. Each row stores the block hash, record hash, event, reference ID, group ID, chain ID, contract address, transaction hash, Sepolia block number, status, any error message and the time of the last update. A record can therefore be locally attested while its anchor is still pending. This is correct behaviour, because the local ledger is immediate whereas the public blockchain is asynchronous.

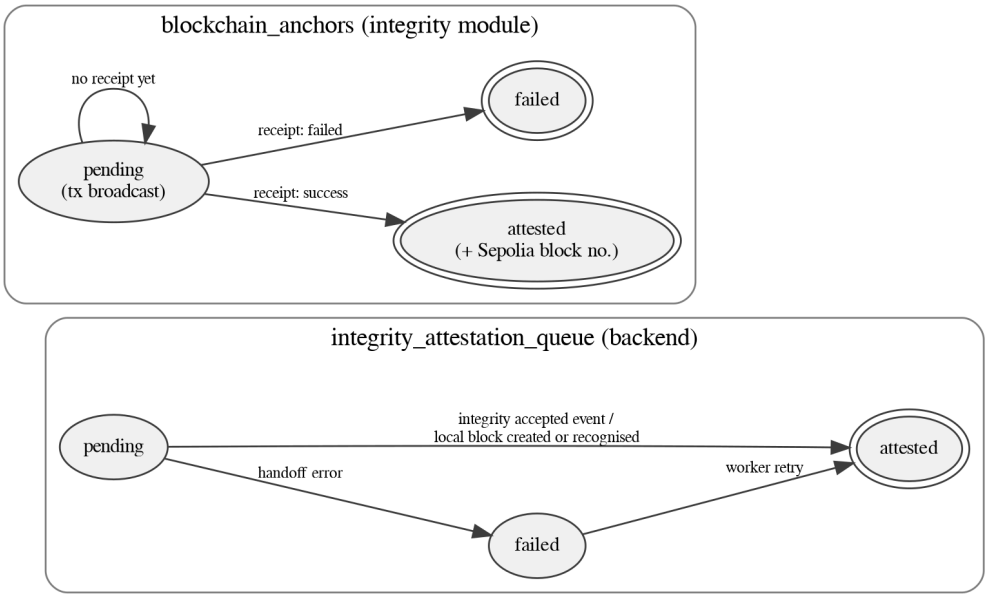


Figure 4.4: State model of the attestation queue and of blockchain anchors

Reconciliation. A background scheduler in the integrity service runs at a configurable interval, set to 60 seconds. It selects pending anchor rows that have transaction hashes and queries Sepolia for their receipts. A successfully mined transaction updates the row to attested and records the Sepolia block number; a failed transaction updates it to failed; and a row without a receipt remains pending. A manual reconciliation endpoint supports demonstration and debugging, but normal operation does not require it.

Verification procedure. Verification checks (1) that each block hash matches its recalculated hash, (2) that each block's previous hash refers to a known block, (3) that each academic event block's record hash matches the hash of the current database snapshot, and (4) that the anchor linked to each academic block can be found on Sepolia. The verify endpoint returns these results as `is_valid`, `local_valid`, `blockchain_valid`, `block_count` and a list of `invalid_blocks`. If a database record is modified after attestation, the recomputed record hash changes and the block is marked invalid, even though its block hash is still structurally correct.

4.5 Standards and professional practice

The design applies recognised standards and professional practices.

- **Cryptographic hashing.** Record and block hashes use SHA-256 as specified in FIPS 180-4 [18].
- **Provenance concepts.** The event model reflects the entity, activity and agent concepts of the W3C PROV data model [17]: records are entities, workflow events are activities, and students and supervisors are agents.
- **Data protection.** Privacy by design and data minimisation guided the off-chain and on-chain split, consistent with the Sri Lankan Personal Data Protection Act No. 9 of 2022 [41] and with the regulatory concerns identified in the literature [8].
- **Smart contract engineering.** The contract is minimal, owner-restricted, non-upgradeable and validates its inputs, following the recommendations that emerge from the smart contract security literature [5], [19], [20], [28], [29].
- **Secret management.** The signing key and RPC credentials are supplied through environment configuration and excluded from source control; transactions are signed locally within the integrity service.
- **Interoperable interfaces.** The integrity service exposes RESTful JSON endpoints, and blockchain access uses the standard Ethereum JSON-RPC interface through an RPC provider.
- **Academic referencing.** IEEE referencing style is used throughout this thesis.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The integrity component is a Python service that runs alongside the other ResearchLens services. Table 5.1 lists the versions declared in its Docker deployment configuration. The Dockerfile installs `requirements-docker.txt`; the separate development `requirements.txt` pins different versions and is not used to build the deployed image. The table therefore records what the deployment configuration declares, not an inspection of the packages installed on the running server.

Table 5.1: Software versions declared by the Docker deployment of the integrity service

Software or technology	Configured version	Purpose
Python	3.11 (python:3.11-slim)	Service runtime; patch version not pinned
FastAPI	0.100.0	Integrity REST API
Uvicorn	0.22.0	ASGI server
Pydantic	2.6.2	Request and data validation
psycopg2-binary	2.9.9	PostgreSQL driver
Web3.py	6.20.1	Sepolia RPC and contract interaction
Solidity compiler	0.8.20	Compiles the anchoring contract
py-solc-x	2.0.4	Solidity compiler integration
Sentence Transformers	2.7.0	Semantic document embeddings
Embedding model	all-MiniLM-L6-v2	Similarity model; revision not pinned
FAISS (CPU)	1.8.0.post1	Vector similarity search
PyTorch	2.3.1+cpu	CPU inference
NumPy	1.26.4	Numerical and vector operations
scikit-learn	1.5.0	Similarity and numerical utilities
pdfplumber	0.11.10	PDF text extraction
pdf2image	1.17.0	PDF-to-image conversion for OCR
pytesseract	0.3.13	Interface to Tesseract OCR
Pillow	12.2.0	Image processing
python-multipart	0.0.9	Multipart file uploads
Requests	2.32.3	HTTP communication
python-dotenv	1.0.1	Environment configuration

Software or technology	Configured version	Purpose
SHA-256	Algorithm, not a release	File, record and ledger hashing
Ethereum Sepolia	Chain ID 11155111	Public testnet anchoring

The rest of ResearchLens uses React with Vite for the frontend, Node.js with Express for the workflow backend and PostgreSQL as the shared database. Anchoring transactions are signed by a MetaMask test account and sent through an Alchemy RPC endpoint, and Sepolia Etherscan serves as an independent explorer.

5.2 Component implementation

Service structure and endpoints. The integrity service exposes the endpoints in Table 5.2 and reads its configuration from environment variables at start-up (Table 5.3).

Table 5.2: Principal endpoints of the integrity service

Endpoint	Function
POST /api/ledger/attest	Reconstructs the snapshot for an event, writes the event block and its main checkpoint, and broadcasts the anchor transaction
GET /api/ledger/verify	Verifies block hashes, previous-hash links, record hashes and Sepolia anchors
GET /api/ledger/dag	Returns DAG nodes, edges and proof-state labels for visualisation
POST /api/blockchain/anchors/reconcile	Runs reconciliation of pending anchors on demand
POST /api/similarity/analyze	Returns a similarity score and decision for an uploaded document

Table 5.3: Configuration items of the integrity service

Configuration item	Purpose
Database URL	Connection to PostgreSQL for snapshots, the ledger and anchor records
Blockchain enable flag	Turns public anchoring on or off
RPC URL	Alchemy endpoint for Ethereum Sepolia
Private key	Owner wallet key used to sign anchoring transactions (secret)

Configuration item	Purpose
Chain ID	11155111 (Sepolia)
Contract address	Address of the deployed anchoring contract
Reconciliation interval	Interval of the background reconciler (60 seconds)

Attestation. When the backend calls the attestation endpoint with an event type and a reference ID, the service rebuilds the canonical snapshot of the fields that matter for that event and hashes it with SHA-256. It then writes two rows to the `blocks` table. The first is the academic event block on the group's branch, and the second is a main checkpoint whose `group_head_hash` ties the new branch head into the overall timeline. If anchoring is enabled, the event block is handed to the blockchain layer, and the endpoint returns as soon as the transaction has been broadcast.

Blockchain layer. A deployment script compiles the contract with py-solc-x (Solidity 0.8.20), deploys it from the owner wallet and saves the ABI artefact. At run time, the service first calls `isAnchored` to avoid paying gas for a block that is already anchored. Otherwise, it builds an `anchorBlock` transaction, signs it locally, broadcasts it and stores a `blockchain_anchors` row with status pending and the transaction hash. The request path never waits for a receipt.

Reconciliation. Every 60 seconds, a background scheduler collects the pending anchor rows that have a transaction hash and asks Sepolia for their receipts. A successful receipt marks the row attested and records the Sepolia block number, and a failed receipt marks it failed. If there is no receipt yet, the row stays pending until the next run. The manual reconcile endpoint runs the same routine on demand.

Verification and views. The verify endpoint walks the ledger. It recomputes every block hash, checks every previous-hash link, rebuilds the live snapshot for every academic block and compares its hash with the stored record hash, and checks that the block's anchor can be found on Sepolia. The DAG endpoint turns the same information into nodes, edges and labels. In the administrator workspace, the Integrity DAG draws the graph, and the Integrity Audit Trail lists each academic event with its Local and Sepolia status, Sepolia block number, submission time,

transaction link and a "Copy proof" action. The trail can be searched, filtered by event type and status, and exported to CSV.

Document checks. Two checks protect uploaded documents. The first is exact: the SHA-256 hash of the uploaded file is compared with the hashes of earlier submissions, and an identical file is refused with the message "This proposal file was already submitted". The second is semantic. Text is extracted with pdfplumber, with pdf2image and pytesseract available for OCR, and is embedded with the all-MiniLM-L6-v2 model. It is then compared with earlier submissions through a FAISS index, excluding documents from the same group or user. The service returns a similarity score and a decision, such as accepted or needs manual review. These checks inform the supervisor's review and are kept separate from the integrity proof.

5.3 Integration with the other components

Backend. The Node.js backend still owns the academic workflow. When a significant event happens, it saves the record and adds an item to the `integrity_attestation_queue`. The queue enforces uniqueness on the event and reference ID, and a worker processes pending and failed items in event-time order. Because all blockchain work happens inside the integrity service, the backend knows nothing about contracts, wallets, gas or receipts.

Document submissions. For TAF and proposal uploads, the order of operations is the one stated in the submission form itself: when the user clicks Submit, the backend sends the PDF to the integrity service for similarity analysis, saves the result, and queues ledger attestation. A duplicate file is refused at submission, so nothing is saved or queued for it.

Frontend. The administrator workspace loads group data from the backend and DAG and audit data from the integrity service. The Integrity DAG shows each group as a horizontal branch (Group, Topic, TAF, review outcomes, Proposal, Checklist, Topic Change and Component events), linked by dashed lines to the main checkpoint chain.

5.4 Verification and software testing

Testing was built around the six test cases described in Section 3.3. Each test case gathered evidence from the database, the verify endpoint and the administrative views. Table 5.4 lists them, together with supporting behaviours that the implementation enforces but that were not separately evidenced.

Table 5.4: Test cases and outcomes

ID	Test	Observed outcome	Status
TC1	Register group R26-SE-011 and follow it to Sepolia	Blocks 121 and 122 written; anchor attested in Sepolia block 11576651; verify reports a valid ledger; event shown On-chain	Passed
TC2	Re-upload a proposal PDF that was already submitted	Upload refused: "This proposal file was already submitted"	Passed
TC3	Compare three embedding models; submit a TAF for R26-SE-011	Benchmark recorded (Table 6.2); TAF scored 75.98% and was routed to manual review	Passed
TC4	Change the supervisor of R26-SE-002 directly in the database	Topic block of R26-SE-002 marked Invalid while its Sepolia proof remains On-chain	Passed
TC5	Change one character of the hash of block 45	Block 45 Invalid and Missing on-chain; the next block in the branch (47) Invalid; later blocks unaffected	Passed
TC6	Change one character of an anchor's block_hash	TAF of R26-SE-004 shown Valid locally but Not anchored	Passed
S1	Anchor the same block hash twice	Skipped by the isAnchored pre-check; rejected by the contract	Enforced by design; not separately evidenced
S2	Anchor from a non-owner address	Rejected by the contract's owner check	Enforced by design; not separately evidenced
S3	Integrity service unavailable during a submission	Queue item remains pending or failed and is retried	Enforced by design; not separately evidenced
S4	Resubmit a document from the same group	Same-group documents excluded from similarity matching	Enforced by design; not separately evidenced

The contract was not run through static analysis tools such as Slither, Mythril or SmartCheck. It is small, makes no external calls, handles no payments and only accepts calls from its owner, all of which limit its attack surface. Even so, static analysis should be applied before any production deployment [19], [28].

5.5 Deployment and operation

The integrity service is packaged as a Docker image built from python:3.11-slim with the dependencies in `requirements-docker.txt`. It is deployed on a virtual private server and published at integrity.researchlens.tech, next to the other ResearchLens services and the shared PostgreSQL database. The contract runs on Sepolia (chain ID 11155111), the owner wallet is a MetaMask test account funded with test Ether, and all chain access goes through Alchemy.

Day-to-day operation needs no manual steps. The backend worker hands events to the integrity service, the service attests them and broadcasts anchors, and the reconciler confirms them every 60 seconds. Administrators can watch the outcome in the Audit Trail. Rows marked "Chain failed" or still pending after an unusually long time are the ones that need attention, and the manual reconcile endpoint helps when investigating them. Operators are also responsible for protecting and, if necessary, rotating the private key, keeping the owner account funded, and recording which contract address was used in each academic period.

6 RESULTS AND VALIDATION

6.1 Research results

Table 6.1 summarises how the design and its validation answer each research question.

Table 6.1: Summary of results for each research question

RQ	Result	Evidence
RQ1	Lifecycle events can be modelled as typed, reference-identified blocks on group branches, each tied into a main checkpoint chain. This gives both an overall timeline and per-group provenance.	Section 4.4; TC1 (blocks 121 and 122); Figure 6.1
RQ2	Rebuilding the canonical snapshot and comparing its hash with the attested record hash exposes direct database edits. Hash links expose edits to the ledger itself.	TC4 and TC5; Figures 6.5 and 6.6
RQ3	Anchoring only hashes and minimal identifiers yields public proof that is independent of the institution. Breaking the link to that proof is itself detected.	TC1 (Sepolia block 11576651); TC6; Figure 6.7
RQ4	Separating local attestation from asynchronous broadcast and reconciliation keeps users responsive while every anchor's state remains tracked.	Table 6.3; TC1 timing

6.2 Product and technical results

TC1: success flow. A test student registered group R26-SE-011 through the ResearchLens interface, and the group row was created in `research_groups` at 08:19:36 on 27 August 2026. The ledger then contained two new blocks for the group. Block 121 holds the group_registration event, with `chain_scope` set to group and the record hash of the new row. Block 122 is a main checkpoint whose `group_head_hash` matches the hash of block 121 (8dc14f9f…). The anchor row for the event reached the attested state with transaction hash 0x577f5969…cc9673 in Sepolia block 11,576,651, and the row was last updated at 08:21:51. In this instance, therefore, about 2 minutes 15 seconds passed between the creation of the record and the recording of its public confirmation, while the student's registration had completed immediately. The public verify endpoint returned the following:

{"is_valid":true,"local_valid":true,"blockchain_valid":true,"block_count":125,"invalid_blocks":[]}

The Integrity DAG (Figure 6.1) showed R26-SE-011 as a branch of Group, Topic, TAF and Component nodes, alongside the branches of other test groups. Most academic nodes were labelled On-chain. Some were labelled Chain failed, for example the topic of R26-SE-008, the group nodes of R26-SE-009 and R26-SE-010, and several later nodes of R26-SE-011. These events were valid locally, but their anchoring transactions had not succeeded at the time of capture. The cause of these failures was not diagnosed within this evaluation. What matters for the design is that they were reported openly rather than hidden or shown as confirmed.

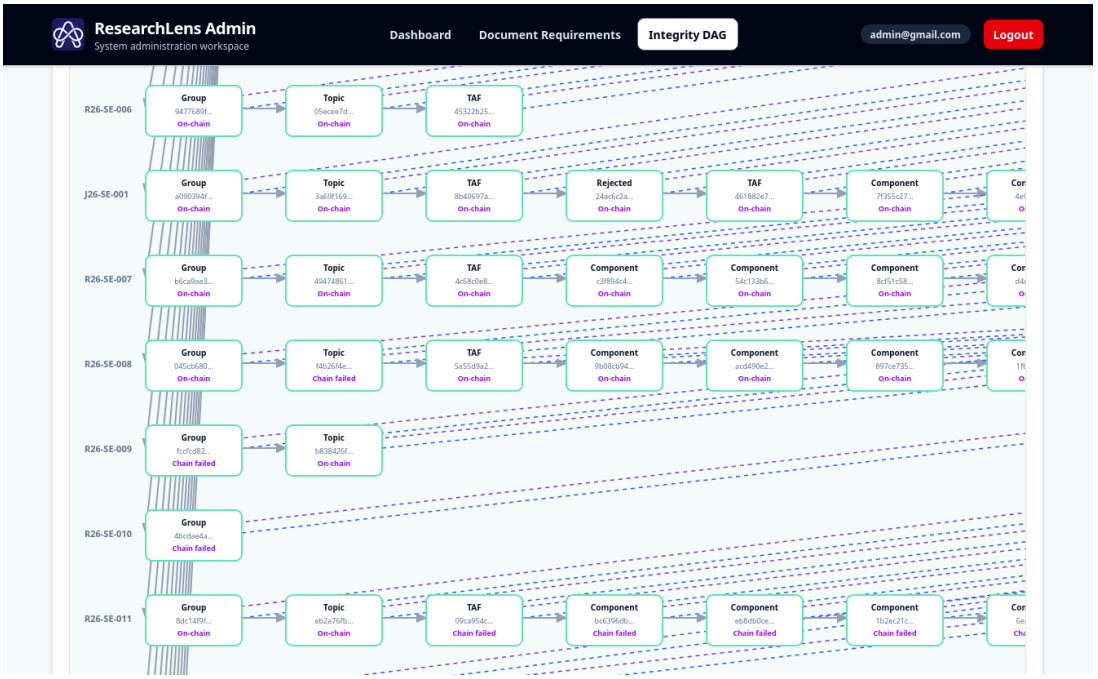


Figure 6.1: Integrity DAG in the administrator workspace, showing group branches and the proof state of each event (TC1)

The Integrity Audit Trail (Figure 6.2) showed the same events as rows with Local and Sepolia status and Sepolia block numbers. Several anchors could share a block. For example, three Component events of R26-SE-004 were recorded in Sepolia block 11,590,742, which shows that anchors for different events can be mined together.

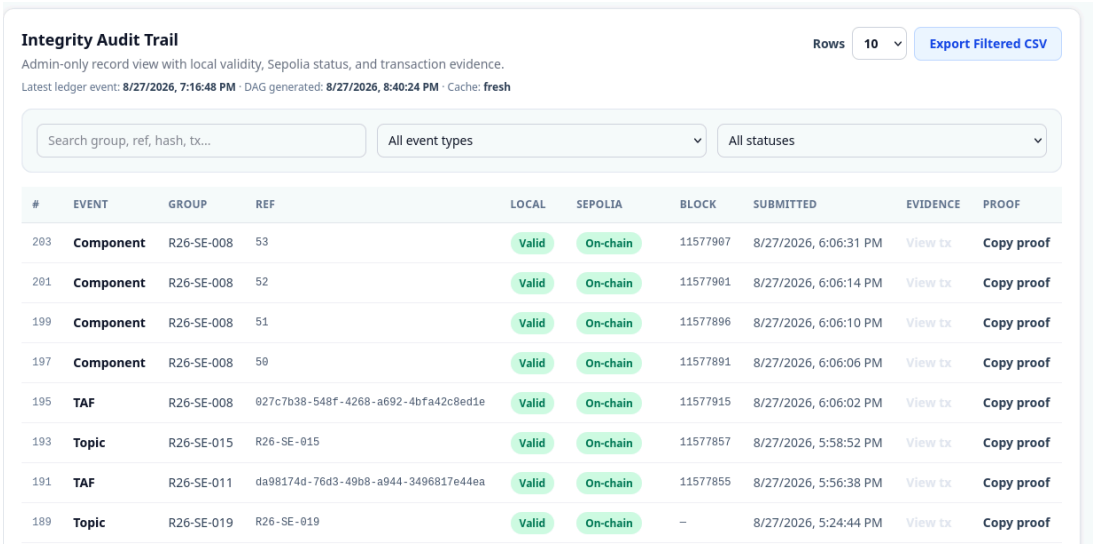


Figure 6.2: Integrity Audit Trail with local validity, Sepolia status and block numbers per event

TC2: duplicate-file prevention. When an already submitted PDF (test_taf_01.pdf) was uploaded again in the proposal form, the submission was refused with the message "This proposal file was already submitted" (Figure 6.3). Because the submission was refused, no record was saved and no attestation was queued, so the duplicate never reached the ledger. This shows that document integrity goes beyond tracking statuses: the service recognises the content of the file itself.

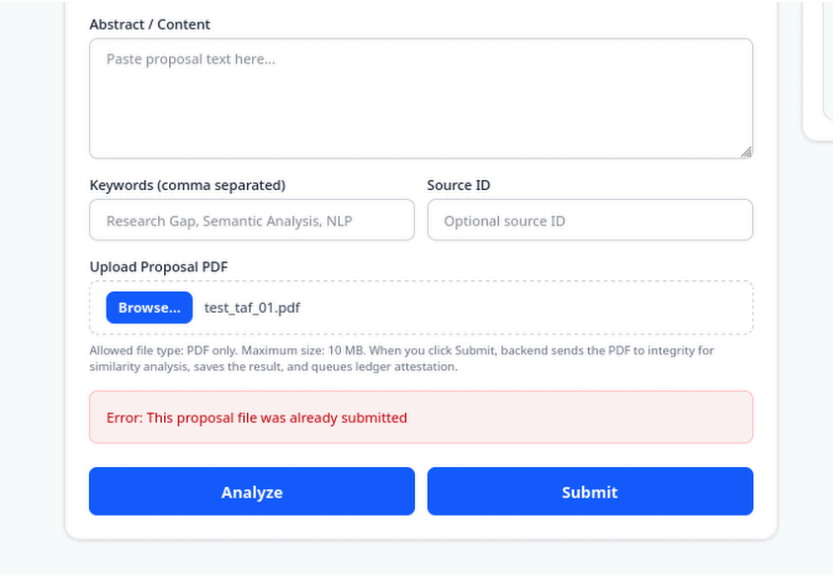


Figure 6.3: Rejection of a previously submitted proposal file (TC2)

TC3: semantic similarity. Exact hashing cannot recognise a document that has been reworded or resaved. To choose a model for semantic checking, three Sentence Transformers models were run on the same corpus of 22 documents (231 pairs) with a threshold of 0.8. Table 6.2 shows the results.

Table 6.2: Comparison of sentence-embedding models on the 22-document corpus

Model	Pairs at or above 0.8	Max score	Mean score	Load time (s)	Encode time (s)
all-MiniLM-L6-v2	3	0.965	0.331	8.39	0.87
paraphrase-MiniLM-L6-v2	3	0.874	0.402	8.28	0.49
all-mpnet-base-v2	5	0.955	0.309	7.71	11.57

The paraphrase model was the fastest encoder, but it scored unrelated documents higher on average (0.402) and its strongest match was weaker (0.874), so it separated related from unrelated documents less clearly. The mpnet model separated well but took about 13 times longer to encode the corpus than all-MiniLM-L6-v2. all-MiniLM-L6-v2 combined a low mean score and the highest maximum with an encoding time under one second, and it was selected for deployment. These figures come from a single run on a small corpus, so they support the choice for this platform rather than a general ranking of the models.

In the deployed service, a TAF submitted for R26-SE-011 was accepted for processing with status pending, a similarity score of 75.98% and the decision "Needs manual review" (Figure 6.4). The file was not a duplicate, as it passed the exact-hash check, yet it was still flagged as substantially similar to earlier material. This is exactly the case that hashing alone would miss. The decision is left to the supervisor rather than taken automatically.

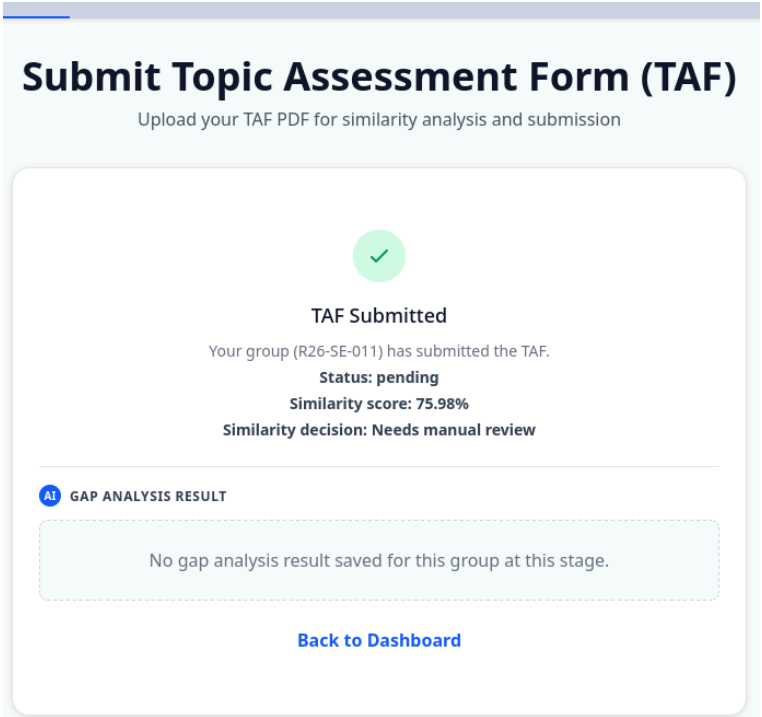
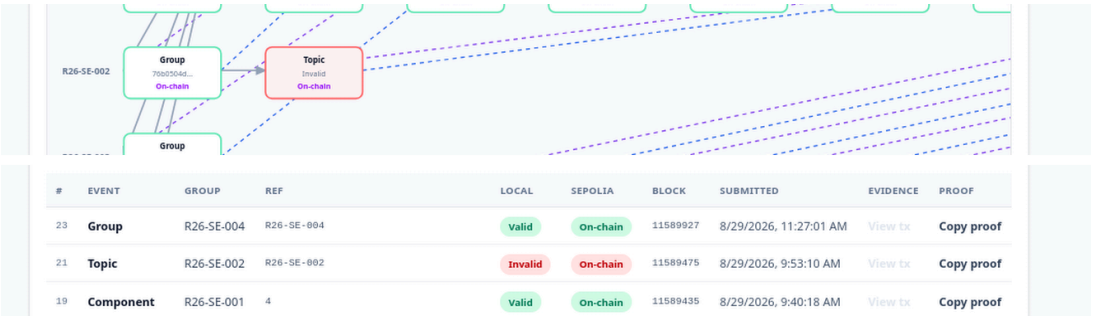


Figure 6.4: TAF submission returned with a similarity score and a manual-review decision (TC3)

TC4: tampering with an academic record. The supervisor of group R26-SE-002 was changed with a direct SQL statement:

```
UPDATE research_groups SET supervisor_id = '<other supervisor id>'
WHERE group_code = 'R26-SE-002';
```

The database reported one row affected, and no application code was involved. After the change, the Topic node of R26-SE-002 was drawn with a red border and labelled Invalid in the Integrity DAG, and audit row 21 showed Local Invalid and Sepolia On-chain (block 11,589,475). The group node remained valid (Figure 6.5). This is the "Invalid / On-chain" state of Table 4.8: the current record no longer matches the attested one, and Sepolia still holds proof of the original hash. The test also shows that the supervisor assignment is among the fields captured in the snapshot of the topic event, so reassigning a supervisor outside the workflow cannot go unnoticed.



#	EVENT	GROUP	REF	LOCAL	SEPOLIA	BLOCK	SUBMITTED	EVIDENCE	PROOF
23	Group	R26-SE-004	R26-SE-004	Valid	On-chain	11589927	8/29/2026, 11:27:01 AM	View tx	Copy proof
21	Topic	R26-SE-002	R26-SE-002	Invalid	On-chain	11589475	8/29/2026, 9:53:10 AM	View tx	Copy proof
19	Component	R26-SE-001	4	Valid	On-chain	11589435	8/29/2026, 9:40:18 AM	View tx	Copy proof

Figure 6.5: Topic block of R26-SE-002 marked Invalid after a direct change of supervisor (TC4)

TC5: tampering with a ledger block. One character of the stored hash of block 45 (the TAF of R26-SE-001) was changed, from …ab485e875… to …ab285e875…:

```
UPDATE blocks SET hash = '<original hash with one character changed>'
WHERE idx = 45;
```

The Audit Trail then showed block 45 as Invalid and Missing on-chain, and block 47 (the next event in the same branch, a topic change) as Invalid but still On-chain. Block 49 and later events remained valid (Figure 6.6). The single edited character therefore broke two things. Block 45 no longer matched its own contents and no longer matched the hash anchored on Sepolia. Block 47 now pointed to a previous hash that no longer existed. An edit to the ledger cannot be made quietly, because it breaks the hash chain from that point onward.

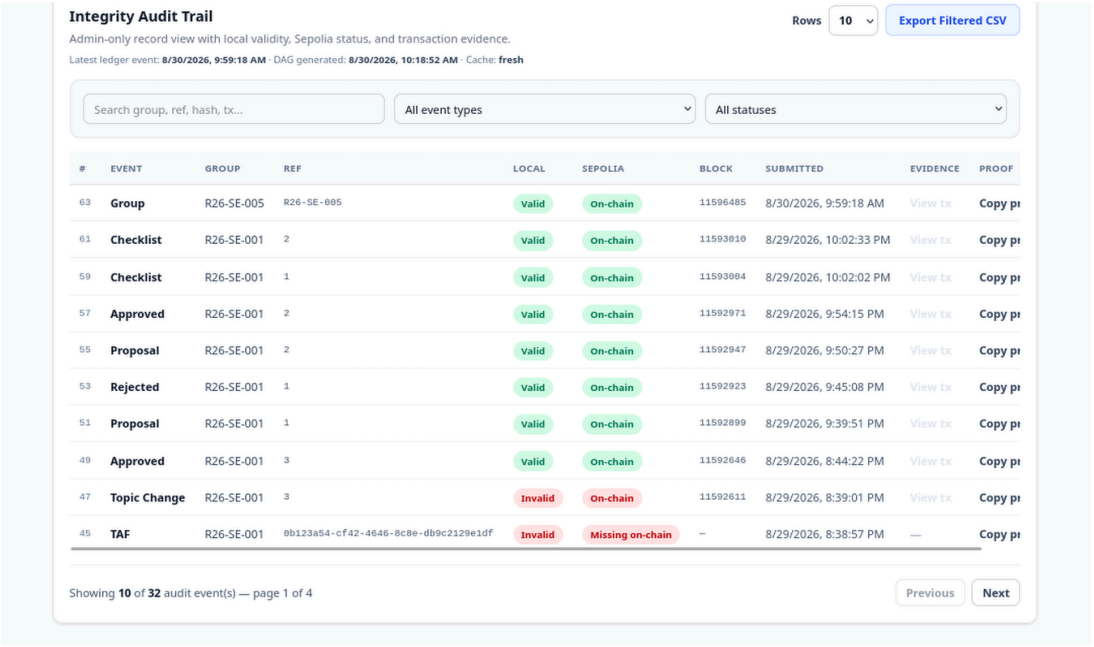
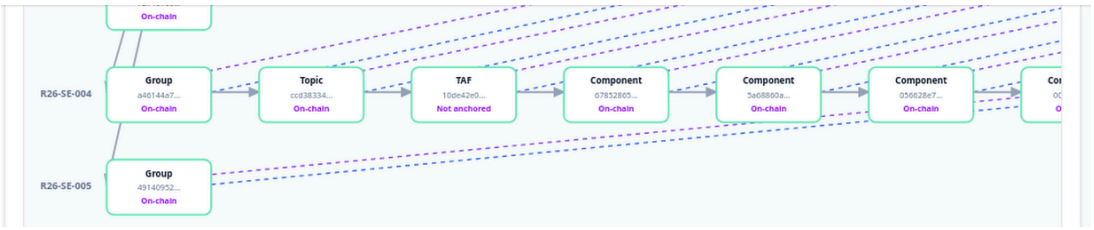


Figure 6.6: Audit Trail after one character of block 45's hash was changed (TC5)

TC6: tampering with the evidence link. One character of the `block_hash` stored in the `blockchain_anchors` row for R26-SE-004's TAF submission was changed, so that the row no longer pointed to the real block. Afterwards, the TAF node of R26-SE-004 was labelled Not anchored in the Integrity DAG, and audit row 27 showed Local Valid and Sepolia Not anchored (Figure 6.7). The local block itself was untouched and still verified. However, the system would no longer claim public proof for it, because the anchor record no longer matched the block. Tampering with the evidence trail therefore lowers the evidence status that is reported, and it does not create a false claim of on-chain proof.



#	EVENT	GROUP	REF	LOCAL	SEPOLIA	BLOCK	SUBMITTED	EVIDENCE	PROOF
31	Component	R26-SE-004	6	Valid	On-chain	11590742	8/29/2026, 2:10:11 PM	View tx	Copy proof
29	Component	R26-SE-004	5	Valid	On-chain	11590739	8/29/2026, 2:10:05 PM	View tx	Copy proof
27	TAF	R26-SE-004	4e8f1f11-ba91-47ce-a2e5-0be60f390866	Valid	Not anchored	–	8/29/2026, 2:09:59 PM	—	Copy proof
25	Topic	R26-SE-004	R26-SE-004	Valid	On-chain	11590703	8/29/2026, 2:07:03 PM	View tx	Copy proof

Figure 6.7: TAF of R26-SE-004 shown as Not anchored after its anchor link was altered (TC6)

Latency behaviour. The earliest implementation waited for the transaction receipt before replying. Some Sepolia transactions confirmed quickly, but others took longer than the 120-second wait that had been allowed, and the interface looked frozen even though the local block already existed. With asynchronous anchoring, the user's request finishes after the broadcast, and confirmation follows in the background, about 2 minutes 15 seconds later in TC1. Table 6.3 compares the two designs.

Table 6.3: Observed behaviour of synchronous and asynchronous anchoring

Stage	Initial synchronous design	Final asynchronous design
Local block creation	Immediate	Immediate
Transaction broadcast	Short delay	Short delay
Public confirmation	Awaited inside the request; sometimes over 120 seconds	Not awaited; handled in the background

Stage	Initial synchronous design	Final asynchronous design
Confirmation update	From the receipt, inside the request	Reconciler every 60 seconds, or manual endpoint
User experience	Interface could appear frozen	Request completes after broadcast

This behaviour agrees with the literature, which reports slow and variable confirmation on Ethereum test networks [4] and latency that rises with load and network size [11], [24]. Gas consumption per anchor and the distribution of confirmation times across many events were not measured and remain future work.

6.3 User or stakeholder validation

No formal user study or structured stakeholder survey was carried out, so this thesis makes no claims about user satisfaction. The administrative views were, however, designed with explanation in mind. The separate Local and Sepolia columns, the "Local checkpoint" label and the interpretation guide in Table 4.8 are meant to let a supervisor or examiner see at a glance what has been proven and what has not. A study with supervisors and administrators is proposed in Section 9.4.

6.4 Integration results

Table 6.4 summarises how the integrity component behaved at each interface with the rest of ResearchLens.

Table 6.4: Integration results

Interface	Integration behaviour	Result
Backend queue to integrity service	Events queued with uniqueness on event and reference ID; worker calls the attestation endpoint	TC1 group registration attested end to end
Integrity service to PostgreSQL	Snapshots rebuilt for attestation and verification; blocks and anchor rows written and updated	Tampering in three different tables detected (TC4–TC6)
Integrity service to Sepolia	Signed transactions broadcast through Alchemy; receipts collected by the reconciler	Anchors confirmed with Sepolia block numbers; failed transactions shown as Chain failed

Interface	Integration behaviour	Result
Admin frontend to integrity service	Integrity DAG and Audit Trail load DAG and audit data	States On-chain, Chain failed, Not anchored and Invalid all displayed
Backend to document checks	PDFs sent for duplicate and similarity checks before attestation	Duplicate refused (TC2); similar TAF routed to manual review (TC3)

6.5 Summary of findings

All six test cases behaved as the design predicts. A new group was attested locally, tied into the main chain and confirmed on Sepolia, about 2 minutes 15 seconds after creation and without delaying the user. Three different kinds of tampering were each detected and reported in a way that matches what actually happened. An edited academic record became Invalid but kept its On-chain proof of the original. An edited ledger block broke its own validity, its on-chain match and the link from the next block. An edited anchor link removed the claim of public proof. On the document side, exact duplicates were refused, and a non-identical but similar TAF was sent for manual review. The main gaps in the evidence are quantitative: gas cost, confirmation-time distributions and verification time as the ledger grows were not measured. In addition, some anchoring transactions failed during testing and were not diagnosed.

7 DISCUSSION

7.1 Interpretation of findings

The results support the central claim of the thesis: in academic workflow governance, blockchain is most effective not as the primary database but as an external trust anchor for cryptographic evidence produced by a flexible off-chain provenance system. Each layer of the design contributes a distinct guarantee. The relational database and file storage provide efficient, private and changeable storage for academic records. The local DAG ledger provides immediate provenance and tamper detection organised according to the structure of the research lifecycle. The public anchor provides evidence that the institution itself cannot silently rewrite.

The three tampering tests show why each layer is needed. In TC4, the ledger itself was untouched; only the academic record changed. A conventional hash chain would not have noticed, but recomputing the live snapshot did. In TC5, the ledger was edited, and the hash links exposed it immediately, both in the edited block and in the block that followed it. In TC6, neither the record nor the ledger changed, only the pointer to the public proof, and the system responded by withdrawing its claim of on-chain proof rather than repeating it. Each edit produced a distinct, recognisable status, which is what makes the views usable for an investigation rather than just an alarm. The document checks complement this. Exact hashing caught the re-uploaded file in TC2, and the embedding model flagged a different but similar TAF in TC3 (75.98%), a case that no hash comparison could detect. The latency result shows that integrity mechanisms must also be designed with usability in mind. Correct cryptography placed on the critical path of a student submission produced a system that appeared broken, whereas the same cryptography placed behind a broadcast-and-reconcile boundary produced a system that behaved normally while still reaching public confirmation.

Finally, the anchoring failures visible in Figure 6.1 are a reminder that a public testnet is not a dependable service. The design copes with them by reporting "Chain failed" honestly while the local proof remains intact, but the failures themselves still need diagnosis. The separation of queue state and anchor state is also meaningful. It makes explicit that "locally attested" and "publicly confirmed" are different claims

with different timing. Presenting both states to supervisors, through labels such as "Chain pending" and "On-chain", avoids overstating the strength of the evidence available at any moment.

7.2 Comparison with literature baselines and existing solutions

Table 7.1 compares the component with representative provenance systems from the literature. The comparison is qualitative because the systems target different domains and were evaluated with different methods; cells marked "Not reported" indicate that the property was not described in the reviewed material.

Table 7.1: Comparison with representative blockchain provenance systems

System	Domain	Off-chain data	Provenance structure	Confirmation handling	Tamper test against live data
Sun et al. [6]	E-healthcare records	IPFS	DAG genealogy of records	Not reported	Not reported
BESURE [7]	Cloud-assisted eHealth	Encrypted cloud storage	Provenance records with audit trail	Validation about 2.8 min	Not reported
Patil et al. [11]	University cloud storage	AWS S3 and IPFS	Linear transaction history	Add latency grows with load and nodes	Not reported
Abhishek et al. [13]	Cloud provenance	IPFS	Linear transaction history	PoS consensus to reduce latency	Not reported
Zhuang et al. [10]	Health information exchange	Records held at facilities	Linkage of record pointers	Not reported	Not reported
This work	Academic research workflow	PostgreSQL and file storage	Group-wise DAG with main checkpoint chain	Asynchronous broadcast and reconciliation	Record, block and anchor-link edits detected (TC4–TC6)

The comparison highlights three distinctions. First, the component is, to the best of the author's knowledge from the reviewed literature, the only one of these designs that models academic lifecycle events, such as topic registration, TAF review,

approvals and topic change requests, as first-class provenance events. Second, while Sun et al. also use a DAG, their DAG represents the genealogy of correlated medical records [6], whereas the DAG in this work represents the concurrent progress of research groups relative to an overall timeline. Third, the reviewed systems emphasise throughput, latency or cryptographic security, whereas this work explicitly treats confirmation latency as a usability problem and demonstrates tampering against live data.

The design choices are also consistent with the performance and security literature. The decision not to wait for confirmation agrees with the slow and volatile behaviour reported for Ethereum test networks [4] and with latency growth under load [11]. The minimal, non-upgradeable contract agrees with the evidence that tool coverage is inconsistent and false positives are common, which makes simple contracts easier to assure [20], [29], and with the centralisation risks of upgradeable contracts [5]. The comparison is still limited, because gas cost and confirmation-time distributions were not measured for this work, so no performance claims relative to these systems can be made.

7.3 Research contribution

The research makes the following contributions:

1. A **hybrid integrity governance architecture** for academic research management that combines off-chain workflow storage, a local DAG provenance ledger and public blockchain anchoring, together with a justification of each layer grounded in the literature.
2. A **group-wise DAG provenance model** for the research lifecycle, consisting of a main checkpoint chain and group branches, with typed academic events, previous, checkpoint and anchor edges, and proof-state labels.
3. A **layered tamper detection mechanism** that verifies ledger structure, live database records and the link to public proof, demonstrated by detecting edits to an academic record, a ledger block and an anchor row.

4. An **asynchronous anchoring pattern** that separates synchronous local attestation from public confirmation, with explicit queue and anchor states and background reconciliation.
5. A **synthesis of the literature** on blockchain provenance, platform performance and smart contract security into six gaps relevant to academic governance (Table 2.2).

7.4 Product contribution

From a product perspective, the component adds verifiable integrity to the group's research management platform without changing how students and supervisors use it. Students continue to submit through the normal interfaces and experience no blockchain-related delay. Administrators gain an Integrity DAG and an Audit Trail that show, event by event, what is locally valid and what is publicly proven, with block numbers, transaction links, copyable proofs and CSV export. These views can support the resolution of disputes about submissions, approvals, supervisor assignments and topic changes. Students are protected against duplicate submissions, and supervisors receive a similarity score with a clear manual-review flag rather than an automatic verdict. Because blockchain logic is confined to one service, other components can benefit from integrity guarantees simply by enqueuing events.

7.5 Limitations and threats to validity

Limitations of the implementation. The contract runs on the Sepolia testnet, so production cost, availability and long-term governance were not evaluated. Several anchoring transactions failed during testing (Figure 6.1), and their cause was not diagnosed. Group and reference identifiers are anchored as readable strings, which could reveal metadata in production. The reconciliation interval is fixed, and there is no adaptive retry, replacement of dropped transactions, gas estimation strategy or alerting for long-pending or failed anchors. The owner wallet is a single key, so losing or leaking it would interrupt anchoring until a new key and contract were configured.

Internal validity. The six test cases were run by the developer on test accounts, one case at a time. They show that the mechanisms work as designed, but they would not

reveal faults that appear under concurrent use, unusual data or infrastructure failures. Supporting behaviours S1 to S4 in Table 5.4 are enforced by the implementation but were not separately evidenced.

Construct validity. Detection of a direct database modification is a strong indicator of tamper evidence, but tamper evidence is not tamper prevention. An attacker with full control of the server could suppress verification or replace the verification code, and the public anchor helps only if a verifier compares local data with the on-chain record. The guarantee is therefore strongest when verification can be performed by a party other than the operator.

External validity. The evaluation used one platform, one testnet and a limited set of test groups. The similarity benchmark used 22 documents and a single run, so its timings and scores indicate relative behaviour rather than general performance. Performance and behaviour on main networks, under larger numbers of groups and events, or in other institutions' workflows may differ. Testnet behaviour is known to be volatile [4].

Reliability of literature evidence. Several quantitative findings in Chapter 2 come from studies that used different software versions, workloads and environments, so they indicate trends rather than directly comparable baselines.

7.6 Implications

For institutions. Universities can obtain independent integrity evidence for academic decisions without moving academic data onto a public blockchain or disrupting existing workflows. The same pattern could strengthen trust in assessment, supervision and approval records more generally.

For practitioners. The work illustrates a practical pattern for integrating blockchain into interactive applications: keep business logic off-chain, hash canonical snapshots of records, anchor only proofs, and confirm asynchronously. It also shows the value of presenting proof state honestly in the user interface.

For researchers. The work suggests that provenance research should evaluate tamper detection against live data rather than relying on the immutability of the

ledger alone, and that usability, not only throughput, should be treated as a first-class constraint in blockchain-backed systems.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The value proposition of the component is **verifiable trust in academic records without disruption or exposure**: universities can prove, to students, examiners, accreditation bodies or appeal panels, that the record of a research project has not been altered since it was created, while keeping academic data private and student workflows responsive. The primary target users are higher education institutions that manage undergraduate and postgraduate research projects digitally. Within an institution, the beneficiaries are students, who gain protection of their submissions and approvals; supervisors and examiners, who gain a reliable history of progress and decisions; and research administrators and quality assurance units, who gain auditable evidence for dispute resolution and quality reviews.

8.2 Market and competitive position

Existing research and learning management systems generally rely on database logs and administrative controls for accountability, and dedicated similarity-checking services address originality rather than the integrity of workflow history. Blockchain provenance solutions described in the literature target healthcare, cloud storage and data governance rather than academic research workflows (Section 2.3). The component therefore occupies a niche between conventional research management systems and general-purpose blockchain provenance platforms. Its competitive advantages are its domain-specific event model, its group-wise DAG view designed for supervisors, its privacy-preserving data placement and its non-blocking user experience. Its main competitive risks are that institutions may consider database audit logs sufficient, and that the cost and governance implications of public blockchains may raise concerns.

8.3 Business or sustainability model

Two sustainability routes are feasible. In an **institutional deployment model**, the component is operated by a university as part of its research management platform, with the anchoring wallet funded from the institution's IT budget; costs are limited to hosting the services and paying transaction fees, which are reduced because only

academic event blocks are anchored and checkpoint blocks are not. In a **software-as-a-service model**, the integrity service could be offered to several institutions as a hosted add-on, priced per institution or per active research group, with the provider managing anchoring infrastructure. In either model, transaction costs can be further controlled by choosing an appropriate network, for example a lower-cost public network or a permissioned network operated by a consortium of universities, without changing the off-chain design. An open-source release of the core ledger and contract is another option that would encourage adoption and independent review, with revenue, if required, coming from hosting, integration and support services.

8.4 Scalability intellectual property and compliance

Scalability. The off-chain design scales with the database, since attestation and verification are local operations; public anchoring is asynchronous and therefore does not limit user throughput. Verification time is expected to grow with ledger size, and measuring this growth is planned future work. If transaction volume or cost became significant, multiple block hashes could be aggregated before anchoring, which is a common optimisation in the literature (for example, batching is proposed by D'Antonio and Uccello [8]).

Intellectual property. The component builds on widely used open-source technologies, including FastAPI, Web3.py, py-solc-x, React, Node.js and PostgreSQL, whose licences should be reviewed before commercial distribution. Ownership of the code developed for the project is subject to the university's policies on student work and the group's agreement.

Compliance. The design follows data minimisation by keeping personal and academic data off-chain, which supports compliance with the Personal Data Protection Act No. 9 of 2022 in Sri Lanka [41] and helps avoid the conflict between blockchain immutability and the right to erasure [8]. A production deployment should additionally pseudonymise the identifiers anchored on-chain, complete a data protection impact assessment, and document the retention and verification procedures for anchored proofs.

8.5 Contribution to the Sustainable Development Goals

The component contributes to the United Nations Sustainable Development Goals [42] as summarised in Table 8.1.

Table 8.1: Contribution to the Sustainable Development Goals

Goal	Contribution
SDG 4: Quality education	Protects the integrity of research assessment records, supporting fair and trustworthy evaluation of students' research work
SDG 9: Industry, innovation and infrastructure	Demonstrates a practical, privacy-preserving integration of blockchain into institutional digital infrastructure
SDG 16: Peace, justice and strong institutions	Strengthens accountability and transparency in institutional decision-making by making academic records tamper-evident and independently verifiable

The environmental impact of blockchain use was also considered. Literature shows large differences in energy and carbon cost between platforms and consensus mechanisms [15], [27]. By anchoring only compact proofs, anchoring only academic event blocks and allowing deployment on an energy-efficient network, the design keeps its environmental footprint small.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis addressed the problem of providing tamper-evident provenance and public integrity proof for academic research workflows without storing private academic data on-chain and without blocking users on blockchain confirmation latency. A structured review of 36 records from the literature showed that existing blockchain provenance systems target other domains, focus on file-level rather than event-level accountability, use linear structures that hide multi-group progress, rarely test tamper detection against live data and treat confirmation latency as an acceptable cost.

In response, a hybrid integrity governance component was designed, implemented and integrated into a research management platform. Academic events are converted into canonical record snapshots, hashed with SHA-256 and recorded in a local DAG ledger made up of a main checkpoint chain and group-wise branches. The block hash and record hash of each academic event block are anchored on Ethereum Sepolia through a minimal, owner-restricted and non-upgradeable contract, while all documents and academic data remain off-chain. Local attestation is synchronous and immediate, and public confirmation is handled asynchronously by a background reconciler.

Six test cases on the deployed system confirmed the design. A newly registered group was attested, tied into the main chain and confirmed in Sepolia block 11,576,651 about 2 minutes 15 seconds later, without making the student wait. Edits to an academic record, a ledger block and an anchor link were each detected and shown with a distinct status. An exact duplicate file was refused, and a similar but non-identical TAF (75.98%) was routed to manual review using the all-MiniLM-L6-v2 model, which was selected after a comparison of three embedding models. The work demonstrates that blockchain is most valuable in this setting as an external trust anchor for evidence produced by a flexible off-chain provenance system.

9.2 Achievement of objectives

Table 9.1 summarises the extent to which each objective was achieved.

Table 9.1: Achievement of objectives

Objective	Summary	Status	Evidence
O1	Literature review and gap analysis	Achieved	Chapter 2; Table 2.2; Appendix A
O2	Event-level, group-wise DAG provenance model	Achieved	Section 4.4; TC1; Figure 6.1
O3	Snapshot hashing and layered verification	Achieved	Section 4.4; TC4 and TC5
O4	Minimal anchoring contract on Sepolia	Achieved for testnet	Sections 4.4 and 5.2; TC1 and TC6
O5	Asynchronous anchoring and reconciliation	Achieved	Figure 4.3; Table 6.3; TC1 timing
O6	Integration and validation, including tampering and document checks	Achieved	TC1–TC6; Section 6.4. Gas cost and user evaluation remain open

9.3 Individual research and product contributions

The individual research contributions are the group-wise DAG provenance model for the academic research lifecycle, the two-level tamper detection mechanism based on canonical record snapshots, the asynchronous anchoring and reconciliation pattern with explicit queue and anchor states, and the synthesis of six research gaps for blockchain-based academic governance. The individual product contribution is a deployed integrity governance service for ResearchLens. It comprises the FastAPI module, the Solidity anchoring contract with its deployment and interaction layer, the background reconciler, the duplicate-file and semantic similarity checks, and the Integrity DAG and Audit Trail views used by administrators.

9.4 Future directions

Several directions would extend this work.

- **Quantitative evaluation.** Measure gas per anchor, the distribution of confirmation and reconciliation times over many events, and verification time as

the ledger grows, using repeated trials, possibly with benchmarking frameworks such as Hyperledger Caliper or BCTMark [4], [27]. The similarity benchmark should also be repeated on a larger, labelled corpus so that precision and recall can be reported.

- **Production deployment.** Evaluate deployment on Ethereum mainnet, a layer-2 network or a permissioned Ethereum network, considering cost, privacy and institutional governance.
- **Metadata privacy.** Hash or pseudonymise group and reference identifiers before anchoring.
- **Operational reliability.** Diagnose the anchoring failures observed during testing, retry Chain failed anchors automatically, and add replacement of dropped transactions, gas estimation and alerting for long-pending anchors. Batching or aggregating anchors could also reduce cost.
- **Contract assurance.** Apply static and dynamic analysis tools such as Slither, Mythril or SmartCheck before production deployment [19], [28].
- **Supervisor tooling.** Extend the DAG visualisation with transaction hash links, Etherscan navigation, event-type filtering and exportable audit reports for supervisors and examiners, and evaluate it in a study with supervisors.
- **Contributor role evolution and anomaly detection.** Address the remaining gaps from the review by recording changes in contributor roles over time and by flagging unusual revision, approval or contribution patterns.

REFERENCES

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[3] S. Pongnumkul, C. Siripanpornchana, and S. Thajchayapong, "Performance analysis of private blockchain platforms in varying workloads," in *Proc. 26th Int. Conf. Computer Communication and Networks (ICCCN)*, Vancouver, BC, Canada, 2017, pp. 1–6, doi: 10.1109/ICCCN.2017.8038517.

[4] W. Choi and J. W.-K. Hong, "Performance evaluation of Ethereum private and testnet networks using Hyperledger Caliper," in *Proc. 22nd Asia-Pacific Network Operations and Management Symp. (APNOMS)*, Tainan, Taiwan, 2021, pp. 325–329, doi: 10.23919/APNOMS52696.2021.9562684.

[5] Y. Huang, X. Wu, Q. Wang, Z. Qian, X. Chen, M. Tang, and Z. Zheng, "The sword of Damocles: Upgradeable smart contract in Ethereum," in *Proc. 32nd IEEE/ACM Int. Conf. Program Comprehension (ICPC)*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10556443

[6] L. Sun, D. Liu, Y. Li, and D. Zhou, "A blockchain-based e-healthcare system with provenance awareness," *IEEE Access*, vol. 12, pp. 110098–110112, 2024, doi: 10.1109/ACCESS.2024.3440170.

[7] S. Li, Y. Zhang, C. Xu, N. Cheng, Z. Liu, and X. S. Shen, "BESURE: Blockchain-based cloud-assisted eHealth system with secure data provenance," in *Proc. IEEE/ACM 29th Int. Symp. Quality of Service (IWQOS)*, Tokyo, Japan, 2021, pp. 1–6, doi: 10.1109/IWQOS52092.2021.9521289.

[8] S. D'Antonio and F. Uccello, "Data provenance for healthcare: A blockchain-based approach," in *Proc. IEEE 46th Annu. Computers, Software, and Applications Conf. (COMPSAC)*, Los Alamitos, CA, USA, 2022, pp. 1655–1660, doi: 10.1109/COMPSAC54236.2022.00263.

[9] Y. Zhang, C. Xu, X. Lin, and X. Shen, "Blockchain-based public integrity verification for cloud storage against procrastinating auditors," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 923–937, Jul.–Sep. 2021, doi: 10.1109/TCC.2019.2908400.

[10] Y. Zhuang, L. R. Sheets, Y.-W. Chen, Z.-Y. Shae, J. J. P. Tsai, and C.-R. Shyu, "A patient-centric health information exchange framework using blockchain technology," *IEEE J. Biomed. Health Inform.*, vol. 24, no. 8, pp. 2169–2176, Aug. 2020, doi: 10.1109/JBHI.2020.2993072.

[11] A. Patil, A. Jha, M. M. Mulla, D. G. Narayan, and S. Kengond, "Data provenance assurance for cloud storage using blockchain," in *Proc. Int. Conf. Advances in Computing, Communication & Materials (ICACCM)*, Dehradun, India, 2020, pp. 443–448, doi: 10.1109/ICACCM50413.2020.9213032.

[12] A. U. Rehman, N. Tariq, M. A. Jan, F. Khan, H. Song, and M. Ibrahim, "A blockchain-based hybrid model for IoMT-enabled intelligent healthcare system," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 4, pp. 3512–3521, Jul.–Aug. 2024, doi: 10.1109/TNSE.2024.3376069.

[13] P. Abhishek, Y. Akash, and D. G. Narayan, "A scalable data provenance mechanism for cloud environment using Ethereum blockchain," in *Proc. IEEE Int. Conf. Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*, Nitte, India, 2021, pp. 1–6, doi: 10.1109/DISCOVER52564.2021.9663425.

[14] R. D. Garcia, G. S. Ramachandran, R. Jurdak, and J. Ueyama, "Blockchain-aided and privacy-preserving data governance in multi-stakeholder applications," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 3781–3793, Dec. 2022, doi: 10.1109/TNSM.2022.3225254.

[15] N. Horvat, D. B. Gajić, P. Trifunović, V. B. Petrović, D. Dragan, and V. A. Katić, "Performance evaluation of a distributed ledger-based platform for renewable energy trading," *IEEE Access*, vol. 12, pp. 86013–86033, 2024, doi: 10.1109/ACCESS.2024.3416612.

[16] Darshan R, "A hybrid DAG-based provenance and blockchain anchoring framework for integrity verification in academic research workflows," unpublished manuscript, Dept. of Information Technology, Sri Lanka Institute of Information Technology, Sri Lanka.

[17] L. Moreau and P. Missier, Eds., "PROV-DM: The PROV data model," W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/

[18] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

[19] S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, "Systematic review of security vulnerabilities in Ethereum blockchain smart contract," *IEEE Access*, vol. 10, pp. 6605–6621, 2022, doi: 10.1109/ACCESS.2021.3140091.

[20] A. Dika and M. Nowostawski, "Security vulnerabilities in Ethereum smart contracts," in *Proc. IEEE Int. Conf. Internet of Things (iThings), Green Computing and Communications (GreenCom), Cyber, Physical and Social Computing (CPSCom) and Smart Data (SmartData)*, Halifax, NS, Canada, 2018, pp. 955–962, doi: 10.1109/Cybermatics_2018.2018.00182.

[21] P. Frauenthaler, M. Sigwart, C. Spanring, M. Sober, and S. Schulte, "ETH Relay: A cost-efficient relay for Ethereum-based blockchains," in *Proc. IEEE Int. Conf. Blockchain*, Rhodes, Greece, 2020, pp. 204–213, doi: 10.1109/Blockchain50366.2020.00032.

[22] A. A. Monrat, O. Schelén, and K. Andersson, "Performance evaluation of permissioned blockchain platforms," in *Proc. IEEE Asia-Pacific Conf. Computer Science and Data Engineering (CSDE)*, Gold Coast, Australia, 2020, pp. 1–8, doi: 10.1109/CSDE50874.2020.9411380.

[23] P. M. Abhishek, D. G. Narayan, H. Altaf, and P. Somashekar, "Performance evaluation of Ethereum and Hyperledger Fabric blockchain platforms," in *Proc. 13th Int. Conf. Computing Communication and Networking Technologies (ICCCNT)*, Kharagpur, India, 2022, pp. 1–5, doi: 10.1109/ICCCNT54827.2022.9984288.

[24] X. Chen, K. Nguyen, and H. Sekiya, "On the latency performance in private blockchain networks," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 19246–19259, Oct. 2022, doi: 10.1109/JIOT.2022.3165666.

[25] M. Dabbagh, M. Kakavand, M. Tahir, and A. Amphawan, "Performance analysis of blockchain platforms: Empirical evaluation of Hyperledger Fabric and Ethereum," in *Proc. IEEE 2nd Int. Conf. Artificial Intelligence in Engineering and Technology (IICAIET)*, Kota Kinabalu, Malaysia, 2020, pp. 1–6, doi: 10.1109/IICAIET49801.2020.9257811.

[26] K. Toyoda, K. Machi, Y. Ohtake, and A. N. Zhang, "Function-level bottleneck analysis of private proof-of-authority Ethereum blockchain," *IEEE Access*, vol. 8, pp. 141611–141621, 2020, doi: 10.1109/ACCESS.2020.3011876.

[27] D. Saingre, T. Ledoux, and J.-M. Menaud, "BCTMark: A framework for benchmarking blockchain technologies," in *Proc. IEEE/ACS 17th Int. Conf. Computer Systems and Applications (AICCSA)*, Antalya, Turkey, 2020, pp. 1–8, doi: 10.1109/AICCSA50499.2020.9316536.

[28] S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, "Ethereum smart contract analysis tools: A systematic review," *IEEE Access*, vol. 10, pp. 57037–57062, 2022, doi: 10.1109/ACCESS.2022.3169902.

[29] S. Ji, D. Kim, and H. Im, "Evaluating countermeasures for verifying the integrity of Ethereum smart contract applications," *IEEE Access*, vol. 9, pp. 90029–90042, 2021, doi: 10.1109/ACCESS.2021.3091317.

[30] Z. Li, H. Wu, J. Xu, X. Wang, L. Zhang, and Z. Chen, "MuSC: A tool for mutation testing of Ethereum smart contract," in *Proc. 34th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, San Diego, CA, USA, 2019, pp. 1198–1201, doi: 10.1109/ASE.2019.00136.

[31] E. Zhou et al., "Security assurance for smart contract," in *Proc. 9th IFIP Int. Conf. New Technologies, Mobility and Security (NTMS)*, Paris, France, 2018, pp. 1–5, doi: 10.1109/NTMS.2018.8328743.

[32] Q. Zeng et al., "EtherGIS: A vulnerability detection framework for Ethereum smart contracts based on graph learning features," in *Proc. IEEE 46th Annu. Computers, Software, and Applications Conf. (COMPSAC)*, Los Alamitos, CA, USA, 2022, pp. 1742–1749, doi: 10.1109/COMPSAC54236.2022.00277.

[33] S. Holler, S. Biewer, and C. Schneidewind, "HoRStify: Sound security analysis of smart contracts," in *Proc. IEEE 36th Computer Security Foundations Symp. (CSF)*, Dubrovnik, Croatia, 2023, pp. 245–260, doi: 10.1109/CSF57540.2023.00023.

[34] D. Bacciu and A. Carta, "Sequential sentence embeddings for semantic similarity," in *Proc. IEEE Symp. Series on Computational Intelligence (SSCI)*, 2019, doi: 10.1109/SSCI44817.2019.9002824.

[35] T.-T. Ha, V.-N. Nguyen, K.-H. Nguyen, K.-A. Nguyen, and Q.-K. Than, "Utilizing SBERT for finding similar questions in community question answering," in *Proc. 13th Int. Conf. Knowledge and Systems Engineering (KSE)*, 2021, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/document/9648830

[36] A. M. Agrawal et al., "Author-oriented semantic plagiarism detection using transformer architectures," in *Proc. IEEE Int. Conf. Advanced Computing Technologies (ICACT)*, Tirupati, India, 2025, pp. 401–406, doi: 10.1109/ICACT67549.2025.11351398.

[37] R. Chikkamath, M. Endres, L. Bayyapu, and C. Hewel, "An empirical study on patent novelty detection: A novel approach using machine learning and natural language processing," in *Proc. 7th Int. Conf. Social Networks Analysis, Management and Security (SNAMS)*, Paris, France, 2020, pp. 1–7, doi: 10.1109/SNAMS52053.2020.9336557.

[38] X. Wang, T. Ban, L. Chen, and H. Chen, "Decentralised knowledge graph evolution via blockchain," 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10336523

[39] M. Naragaddi, E. Yalagi, O. Yametkar, D. Shivakale, and S. Bagewadi, "PDF question and answer system using LLM and FAISS," in *Proc. IEEE 4th Int. Conf. Advancement in Technology (ICONAT)*, Goa, India, 2025, pp. 1–5, doi: 10.1109/ICONAT66879.2025.11362602.

[40] R. Verma, B. K. Mishra, A. Raj, V. Garg, and A. Kumar, "Enhancing multi-document retrieval through FAISS vector search and NVIDIA NeMo integration," in *Proc. Int. Conf. Innovations and Emerging Technologies in AI*

& Communication Systems (IETACS), Mohali, India, 2025, pp. 1366–1372, doi: 10.1109/IETACS68750.2025.11385476.

[41] *Personal Data Protection Act, No. 9 of 2022*, Parliament of the Democratic Socialist Republic of Sri Lanka, 2022.

[42] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

GLOSSARY

Anchor - an on-chain record that stores the hash of an off-chain ledger block so that its existence and content at a point in time can be verified publicly.

Attestation - the act of creating a local ledger block that records the hash of an academic event's record.

Block hash - the hash computed over a ledger block's index, timestamp, data payload and previous hash.

Canonical record snapshot - a deterministic representation of the database fields relevant to an event, used as the input to record hashing.

Checkpoint block - a structural block on the main chain that connects group branches to the overall ledger timeline; it is local and not individually anchored.

Gas - the unit used to measure the computational cost of executing transactions and contracts on Ethereum.

Genesis block - the first block of the main chain.

Hash anchoring - storing the hash of off-chain data on a blockchain so that the data can later be checked against the on-chain proof.

Reconciliation - the background process that queries the blockchain for receipts of pending anchor transactions and updates their status.

Record hash - the SHA-256 hash of a canonical record snapshot.

Tamper evidence - the property that unauthorised modification can be detected, as distinct from tamper prevention.

Testnet - a public blockchain network used for development and testing, whose currency has no real financial value.

APPENDICES

The appendices provide supplementary material referred to in the main text.

Appendix A Research instruments or extended results

Table A.1 lists the records extracted during the literature review (Section 2.1), with their publication year, theme and reference number in this thesis.

Table A.1: Records extracted in the literature review

No.	Study	Year	Theme	Ref.
1	Ethereum smart contract analysis tools: a systematic review	2022	Contract security	[28]
2	MuSC: mutation testing of Ethereum smart contracts	2019	Contract security	[30]
3	Security assurance for smart contract	2018	Contract security	[31]
4	Evaluating countermeasures for verifying the integrity of Ethereum smart contract applications	2021	Contract security	[29]
5	The sword of Damocles: upgradeable smart contract in Ethereum	2024	Contract security	[5]
6	Systematic review of security vulnerabilities in Ethereum blockchain smart contract	2022	Contract security	[19]
7	Security vulnerabilities in Ethereum smart contracts	2018	Contract security	[20]
8	EtherGIS: vulnerability detection based on graph learning features	2022	Contract security	[32]
9	HoRStify: sound security analysis of smart contracts	2023	Contract security	[33]
10	ETH Relay: a cost-efficient relay for Ethereum-based blockchains	2020	Contract security	[21]
11	A blockchain-based e-healthcare system with provenance awareness	2024	Provenance	[6]
12	BESURE: cloud-assisted eHealth system with secure data provenance	2021	Provenance	[7]
13	Data provenance for healthcare: a blockchain-based approach	2022	Provenance	[8]
14	Data provenance assurance for cloud storage using blockchain	2020	Provenance	[11]
15	A scalable data provenance mechanism for cloud environment using Ethereum	2021	Provenance	[13]

No.	Study	Year	Theme	Ref.
16	A patient-centric health information exchange framework using blockchain	2020	Provenance	[10]
17	Blockchain-aided and privacy-preserving data governance in multi-stakeholder applications	2022	Provenance	[14]
18	Performance analysis of blockchain platforms: Hyperledger Fabric and Ethereum	2020	Performance	[25]
19	Function-level bottleneck analysis of private PoA Ethereum blockchain	2020	Performance	[26]
20	Performance evaluation of Ethereum and Hyperledger Fabric blockchain platforms	2022	Performance	[23]
21	Performance evaluation of Ethereum private and testnet networks using Hyperledger Caliper	2021	Performance	[4]
22	Performance analysis of private blockchain platforms in varying workloads	2017	Public vs private	[3]
23	Performance evaluation of permissioned blockchain platforms	2020	Public vs private	[22]
24	On the latency performance in private blockchain networks	2022	Public vs private	[24]
25	Blockchain-based public integrity verification for cloud storage against procrastinating auditors	2021	Public vs private	[9]
26	BCTMark: a framework for benchmarking blockchain technologies	2020	Public vs private	[27]
27	Performance evaluation of a distributed ledger-based platform for renewable energy trading	2024	Public vs private	[15]
28	A blockchain-based hybrid model for IoMT-enabled intelligent healthcare system	2024	Public vs private	[12]
29	Sequential sentence embeddings for semantic similarity	2019	Similarity	[34]
30	Utilizing SBERT for finding similar questions in community question answering	2021	Similarity	[35]
31	Research on decentralized distributed knowledge graph scheme based on blockchain	2022	Similarity and ownership	Not cited
32	Decentralised knowledge graph evolution via blockchain	2023	Similarity and ownership	[38]
33	An empirical study on patent novelty detection using ML and NLP	2020	Similarity	[37]
34	Author-oriented semantic plagiarism detection using transformer architectures	2025	Similarity	[36]

No.	Study	Year	Theme	Ref.
35	PDF question and answer system using LLM and FAISS	2025	Document retrieval	[39]
36	Enhancing multi-document retrieval through FAISS vector search and NVIDIA NeMo integration	2025	Document retrieval	[40]

Table A.2 indexes the validation evidence reported in Chapter 6.

Table A.2: Index of validation evidence

Test case	Evidence captured	Where reported
TC1	Group row for R26-SE-011; blocks 121 and 122; attested anchor (Sepolia block 11576651); verify endpoint output; Integrity DAG; Audit Trail	Section 6.2; Figures 6.1 and 6.2
TC2	Proposal form rejecting a previously submitted file	Section 6.2; Figure 6.3
TC3	Benchmark output for three embedding models; TAF submission result (75.98%, needs manual review)	Table 6.2; Figure 6.4
TC4	SQL update of supervisor_id for R26-SE-002; DAG node and audit row showing Invalid / On-chain	Section 6.2; Figure 6.5
TC5	SQL update of the hash of block 45; audit rows 45 and 47	Section 6.2; Figure 6.6
TC6	SQL update of an anchor's block_hash; DAG node and audit row showing Not anchored	Section 6.2; Figure 6.7

Appendix B Academic integrity and AI use disclosure

Table B.1 discloses the use of generative AI and automated tools in preparing this thesis, in accordance with the module policy.

Table B.1: Generative AI and automated tool use

Tool	Purpose	Sections affected	Verification by the author
Graphviz	Rendering the architecture and flow diagrams (Figures 2.1 and 4.1 to 4.4) from the author's design	Chapters 2 and 4	Diagrams compared with the implemented system

The research, design, implementation, testing and conclusions are the author's own. The author takes full responsibility for the accuracy, originality, citations, code, data, analyses and conclusions presented in this thesis.